



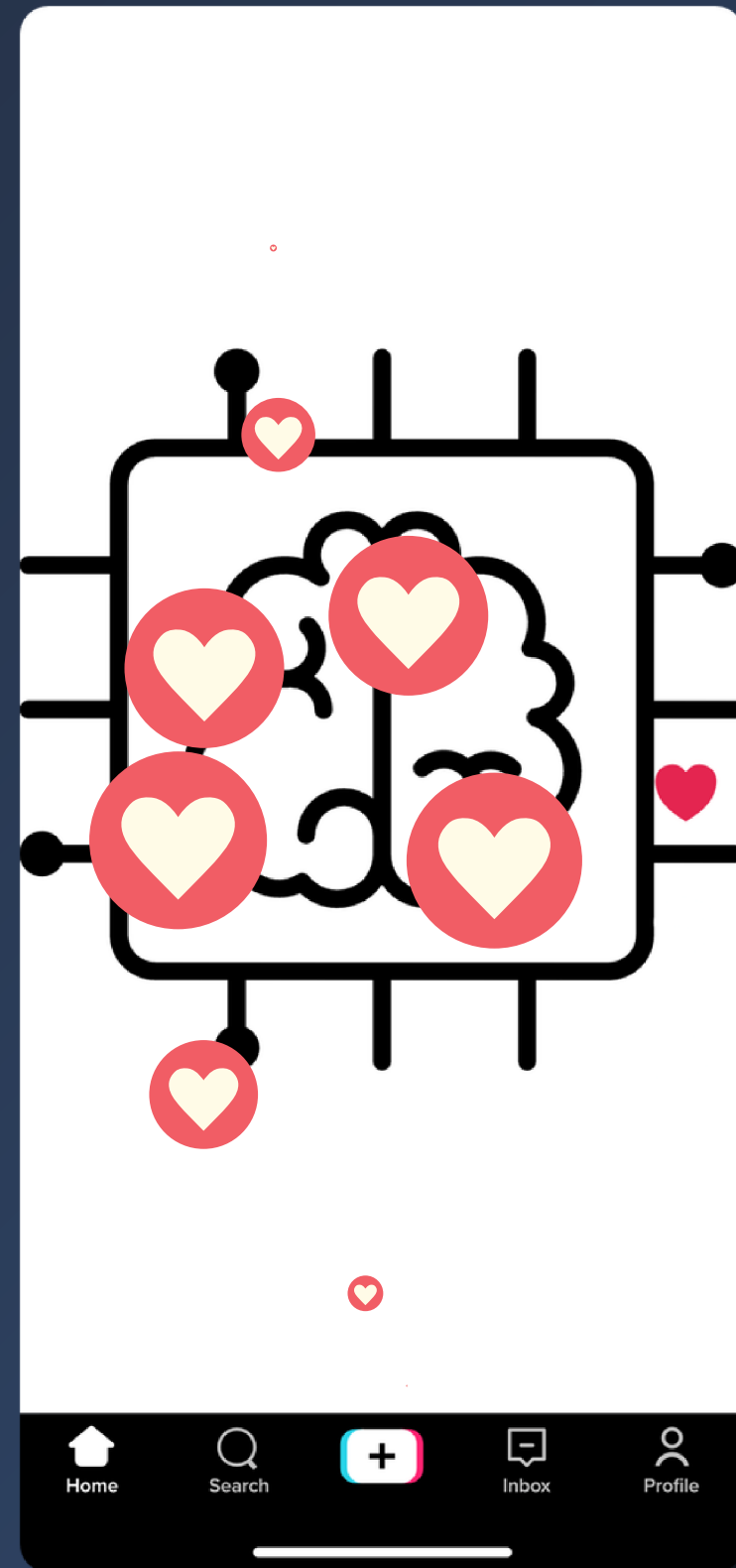
TikTok Creator Intelligence System

NLP-Based Comment Analysis & Content Insights

This project analyzes TikTok comments using Natural Language Processing to help small creators understand audience sentiment, identify key topics, and make data-driven content decisions.

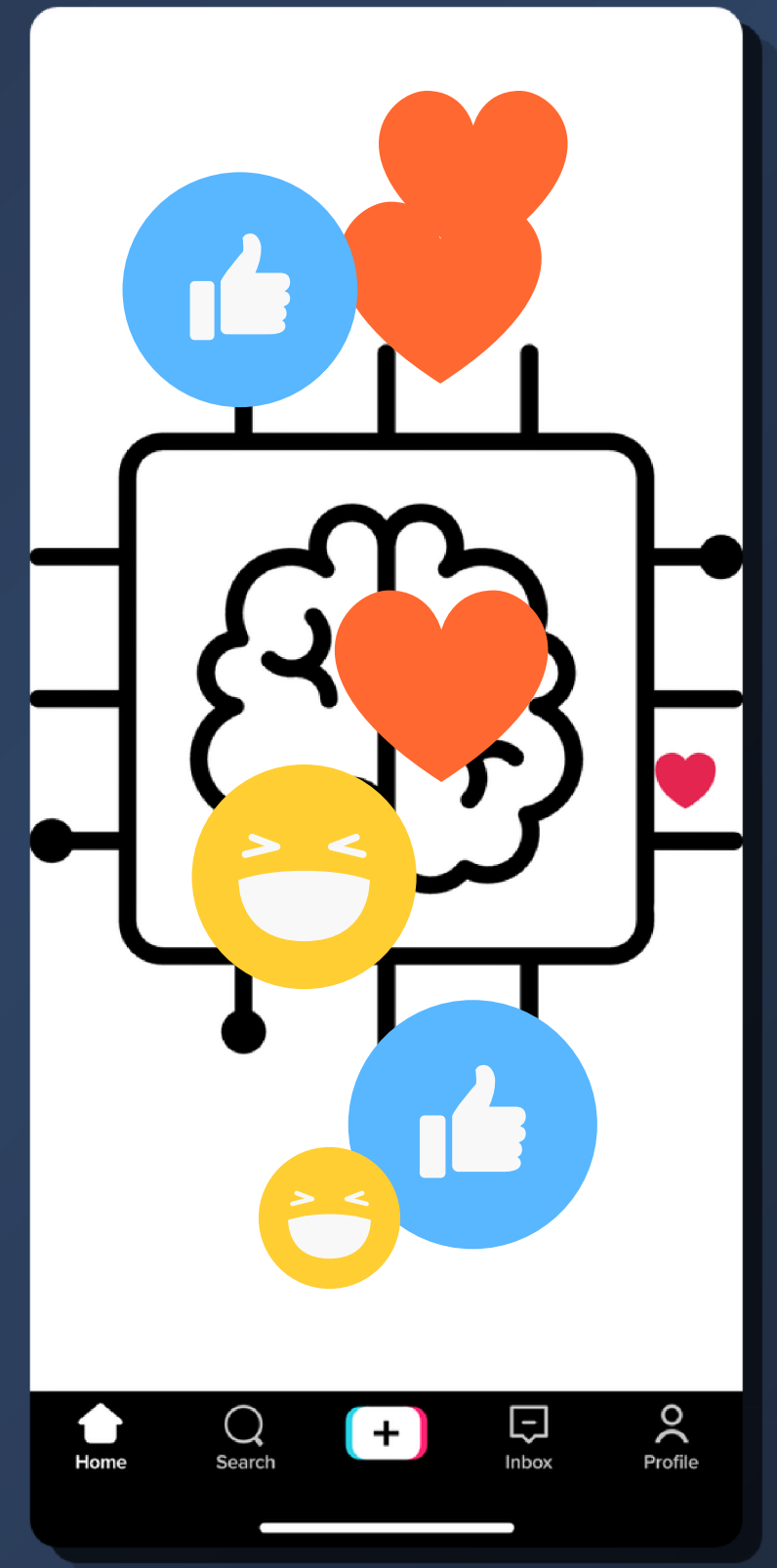
Turning Audience Feedback into Actionable Insights

Presented by Chinelo Lydia Nweke — IBS6 2026



10/04/2026

Deliverable 1 Problem & Goals





Problem Statement

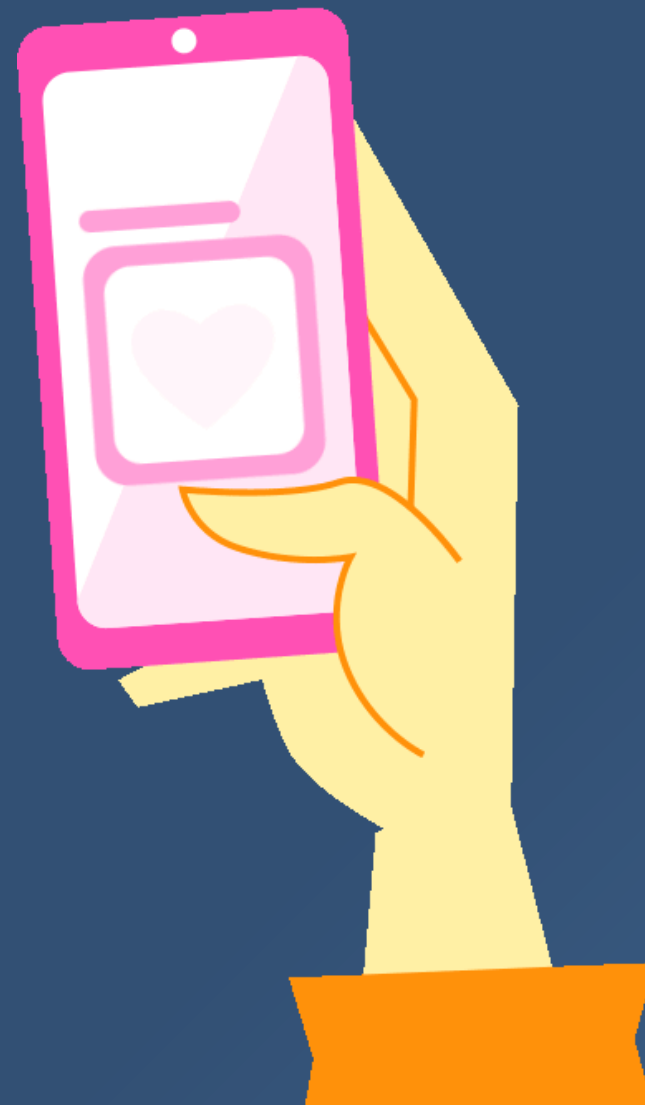
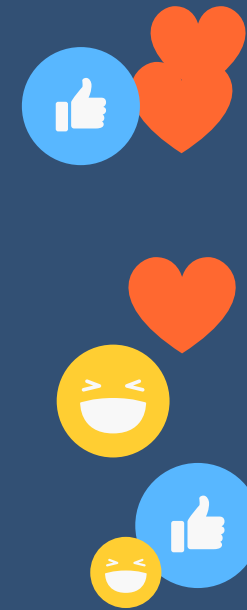
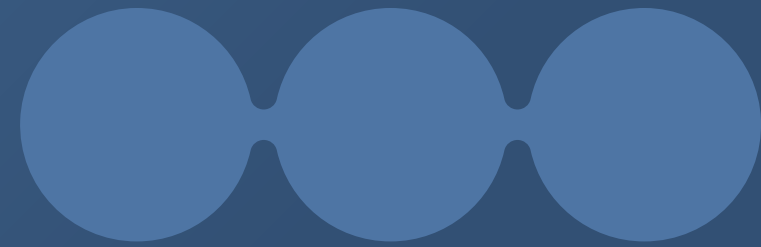
Small TikTok creators struggle to extract meaningful insights from user comments because the feedback is unstructured, scattered, and difficult to analyze manually.



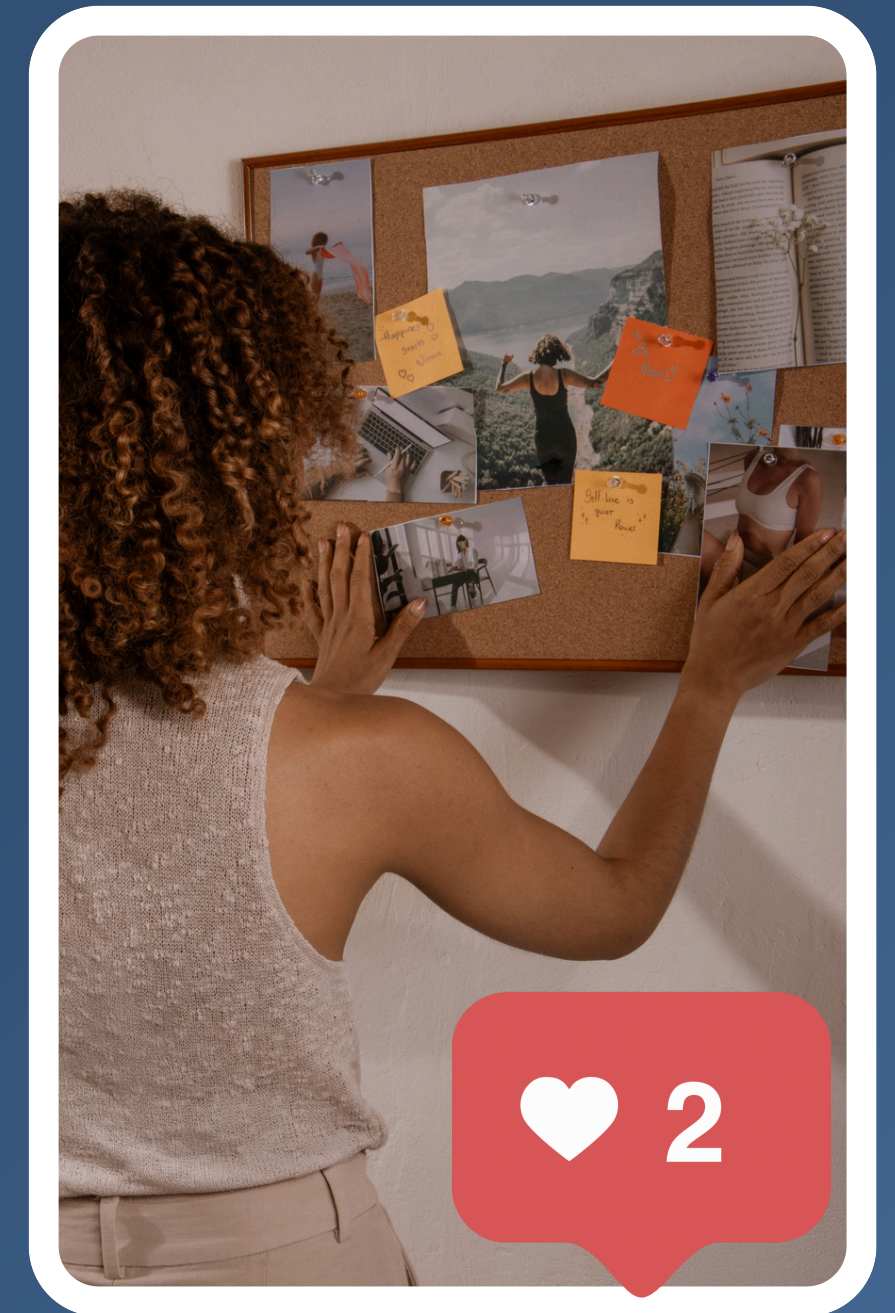


User Description

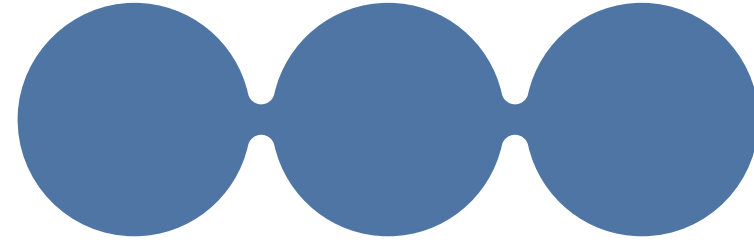
A small TikTok creator (1k–10k followers) who posts content regularly and wants to understand audience reactions but currently relies on manually reading comments and guessing what works.



They also struggle to decide which niche or content direction to focus on because they lack clear insight into what their audience consistently enjoys or engages with.



10/04/2026

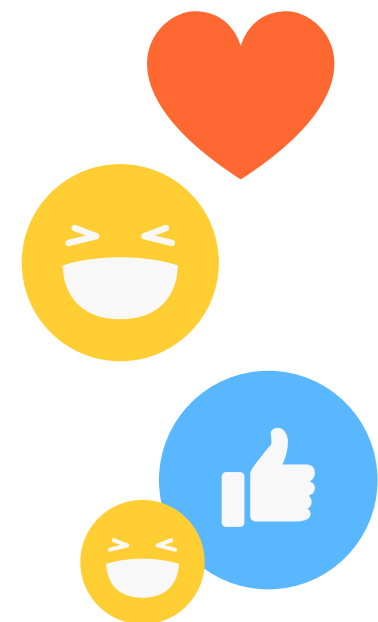
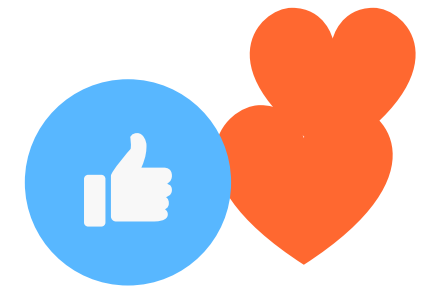


Why This Problem is Relevant

Comments contain valuable feedback such as opinions, questions, and suggestions, but manually analyzing them is time-consuming and inconsistent. As a result, creators miss patterns in audience preferences and lose opportunities to improve content and engagement.



TikTok

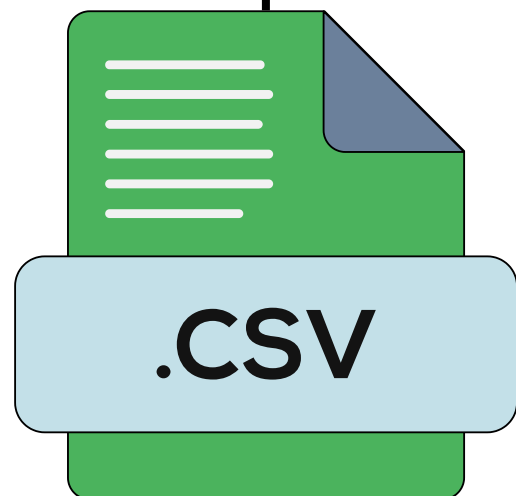


10/04/2026

→ Sketch (Input → NLP Task → Output)

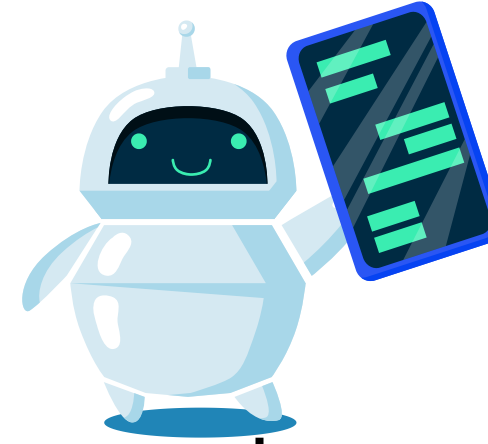
INPUT

CSV file containing TikTok comments and basic video metrics (likes, views, video type)



NLP Task

Text preprocessing
Sentiment analysis (positive, negative, neutral)
Keyword extraction (frequent topics and terms)



OUTPUT

- Sentiment distribution (e.g., 70% positive)
- Key topics discussed in comments
- Simple insights (what users like or complain about)
- Basic recommendations for content improvement



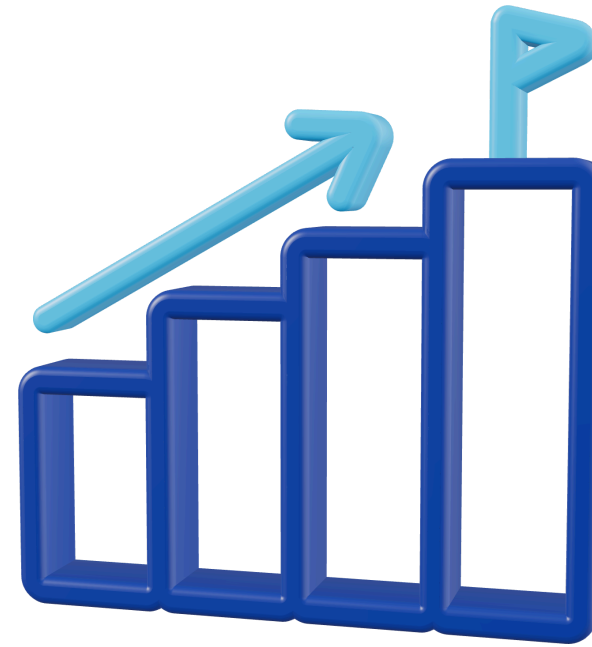


SMART Goals



Main Goal

By July 2026, develop a web-based NLP prototype that analyzes TikTok comments from a CSV file and generates sentiment summaries, keyword insights, and basic content recommendations within a few seconds for datasets of up to 300 comments.



Sub-Goals

Implement a preprocessing pipeline to clean and normalize comment text
Build a sentiment analysis module that classifies comments into positive, negative, and neutral
Develop a simple web interface (Streamlit) for uploading data and displaying insights



Non-Goals

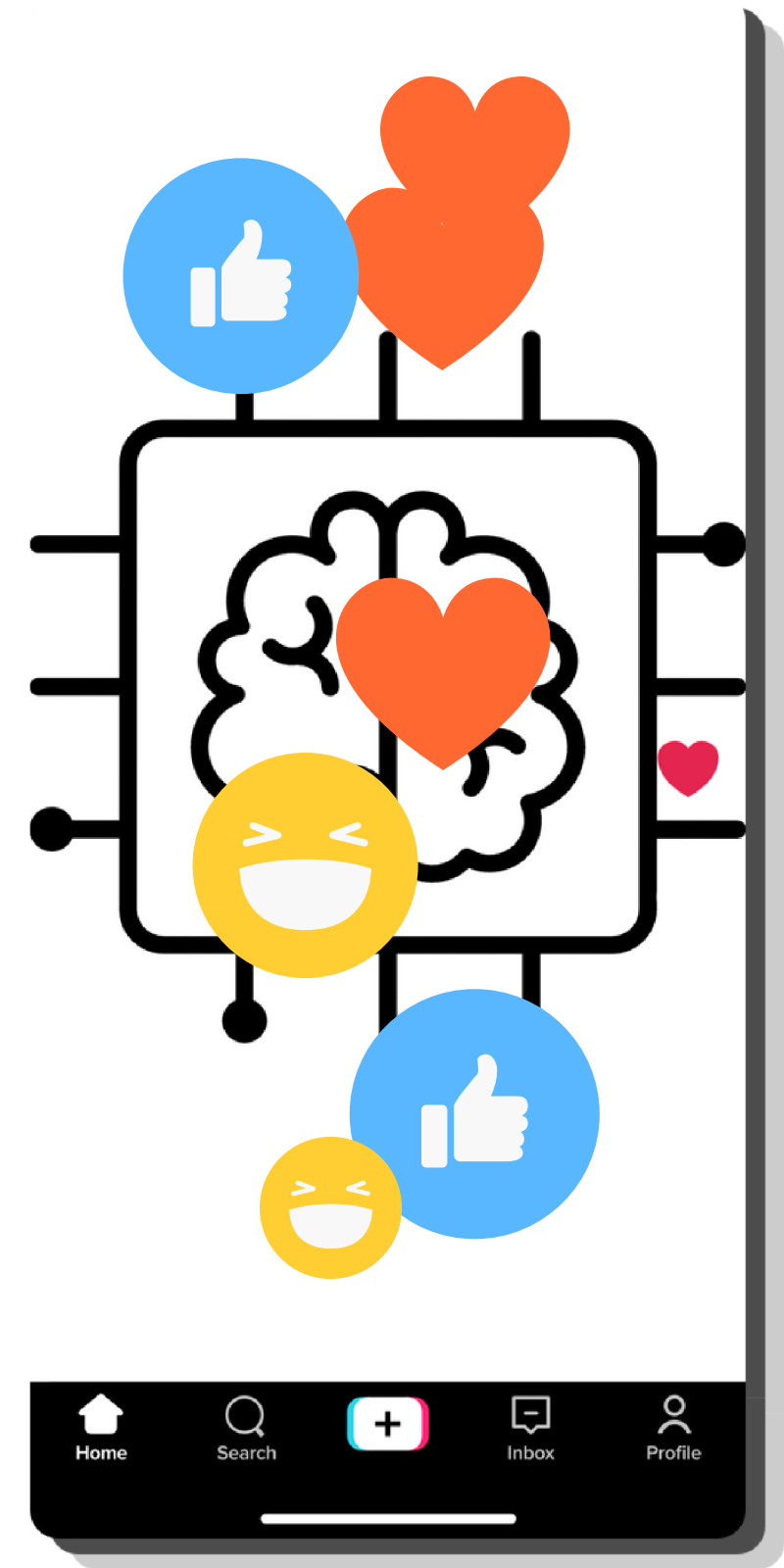
No integration with TikTok API (data will be provided via CSV)
No training of custom deep learning models
No mobile application development

10/04/2026



Deliverable 2

State of the Art

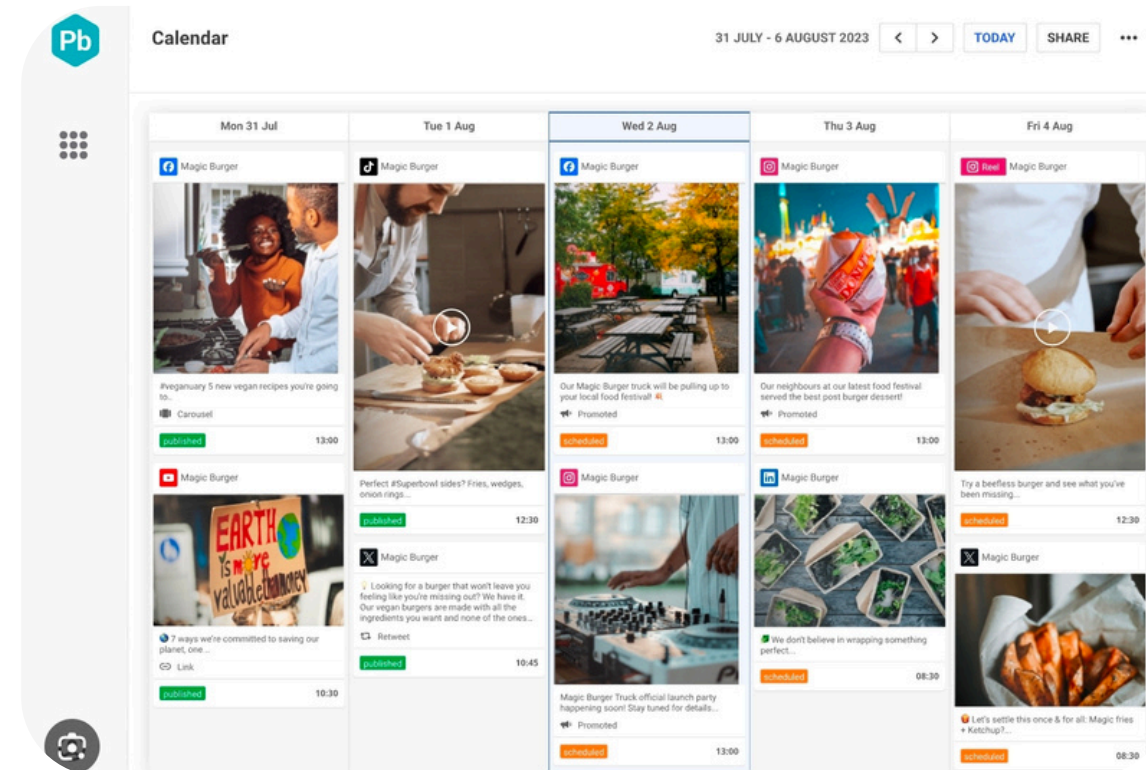




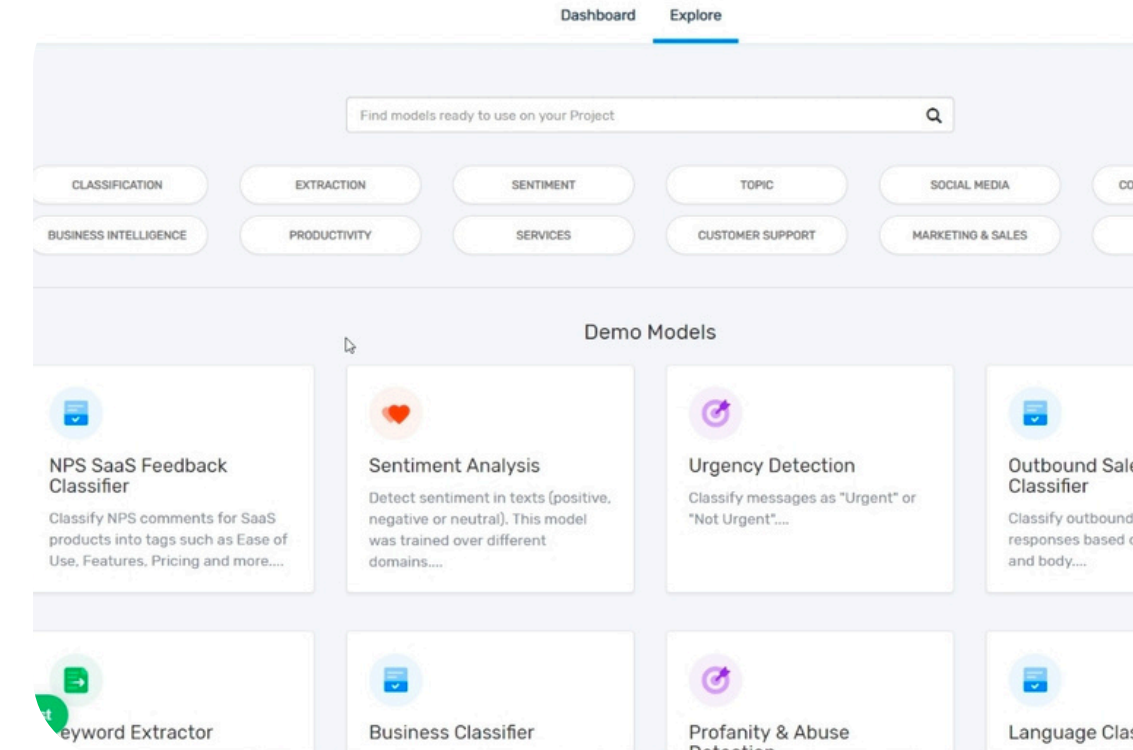
Existing Products / Prototypes



Hootsuite Insights
Social media monitoring and
sentiment analysis
Tracks audience reactions and
trends



Brandwatch
AI-based sentiment and trend
analysis
Large-scale social media analytics



MonkeyLearn
Sentiment analysis and keyword
extraction
General-purpose NLP tool



Limitations of Existing Solutions

- ✓ Designed for large companies, not small creators
- ✓ Expensive and not easily accessible
- ✓ Not tailored to TikTok-specific content
- ✓ Focus on data visualization, not actionable insights
- ✓ Do not connect comments directly to content strategy





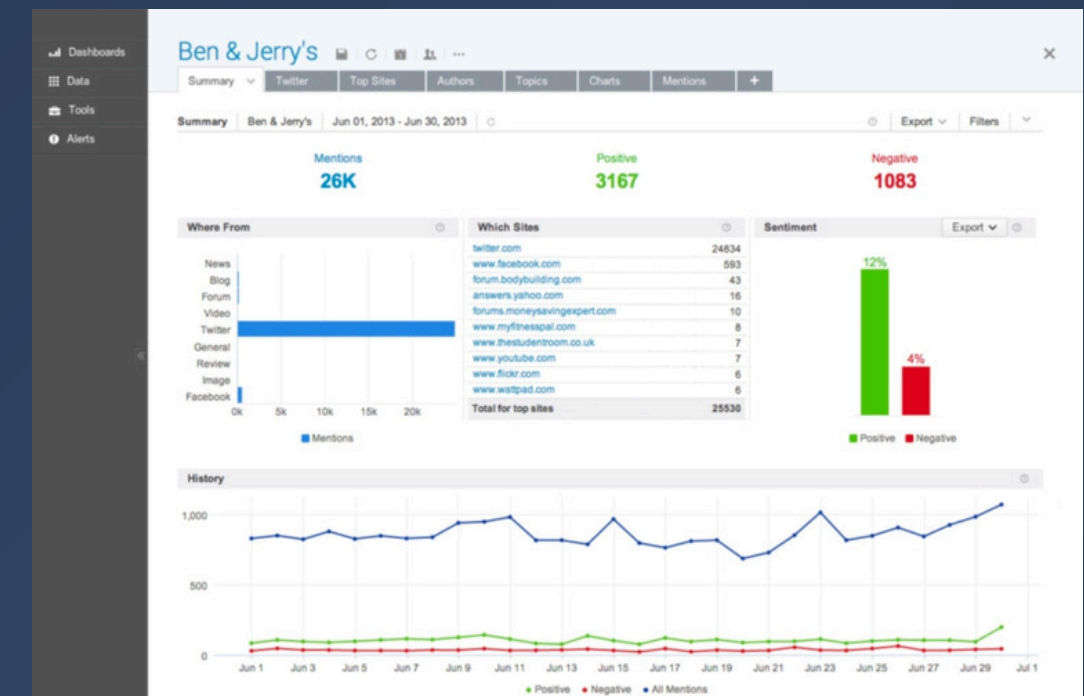
Reverse Engineering



Methods Used:
Sentiment Analysis (text classification: positive, negative, neutral)
Keyword Extraction (TF-IDF, frequency-based methods)



Models / Tools:
BERT (for text understanding)
spaCy / scikit-learn (NLP processing)



Interface:
Web dashboards for visualization and interaction

Parts I Can Reuse

- Text preprocessing pipeline (cleaning and normalization)
- Sentiment classification methods
- Keyword extraction techniques
- Basic dashboard structure for displaying results



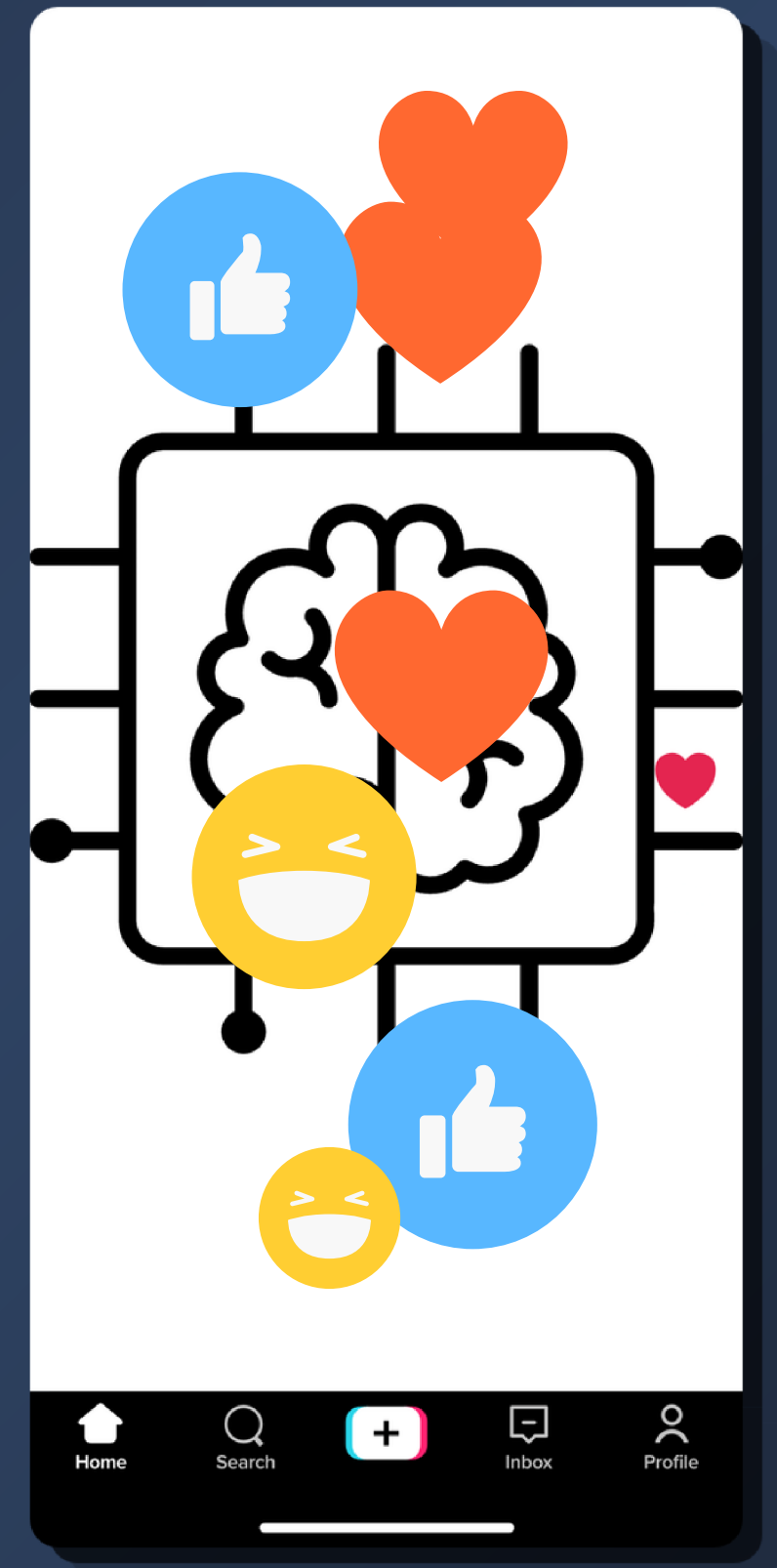
My Contribution (Delta)

- Focus on small TikTok creators (1k–10k followers)
- Simple and accessible system using CSV input
- Links comments to individual video performance
- Generates actionable insights, not just data
- Helps creators decide what content and niche to focus on

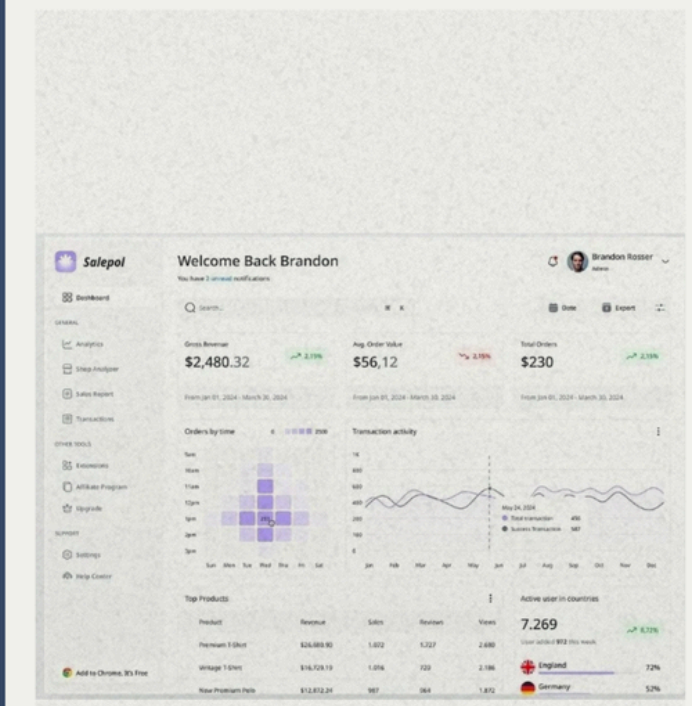
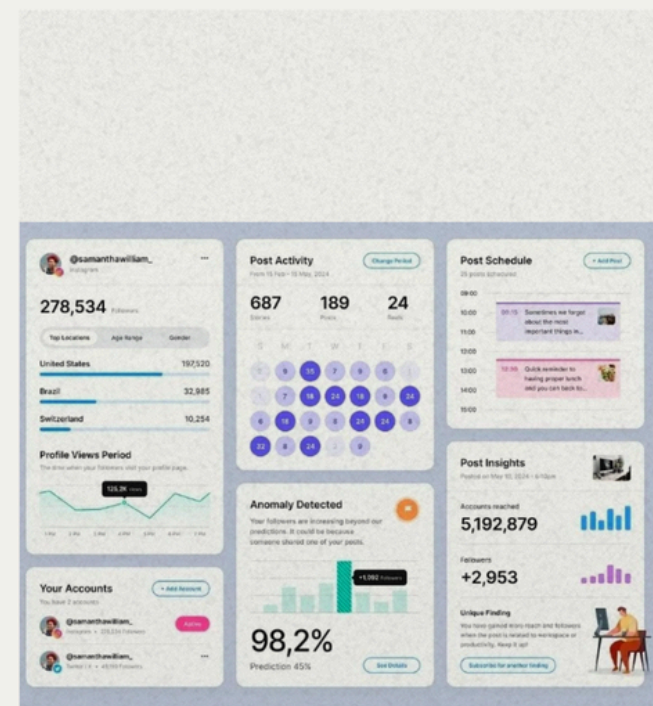
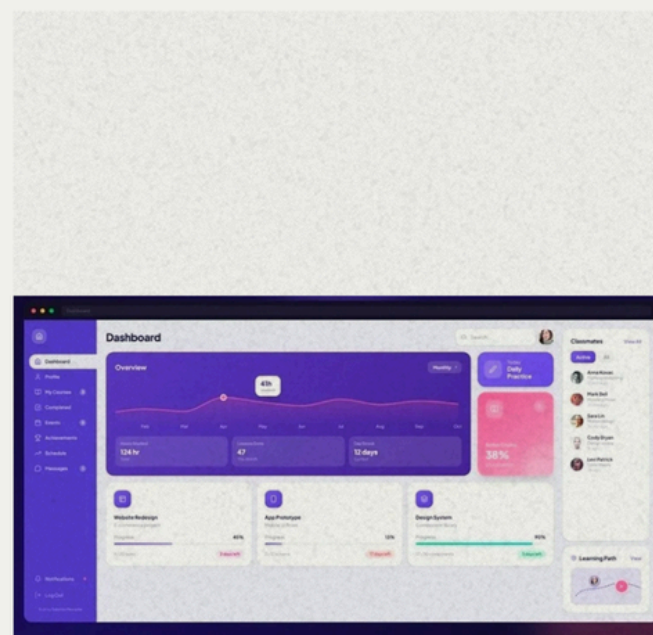
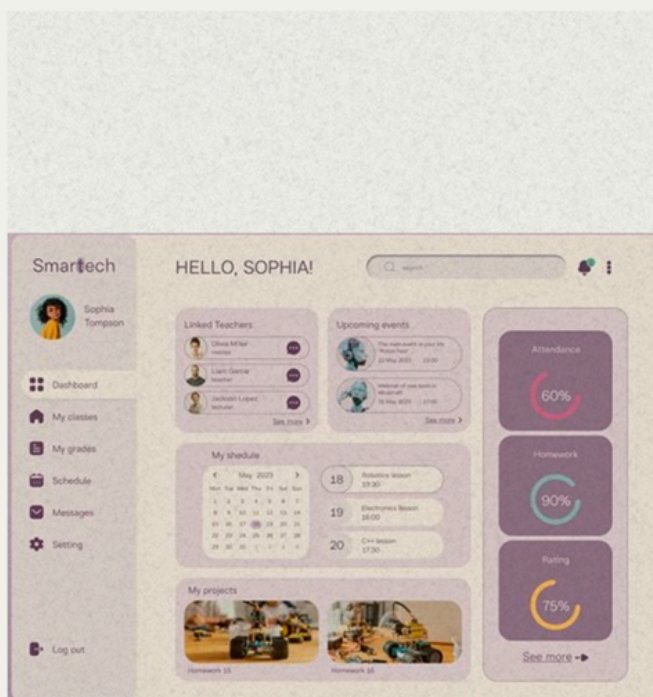


Deliverable 3

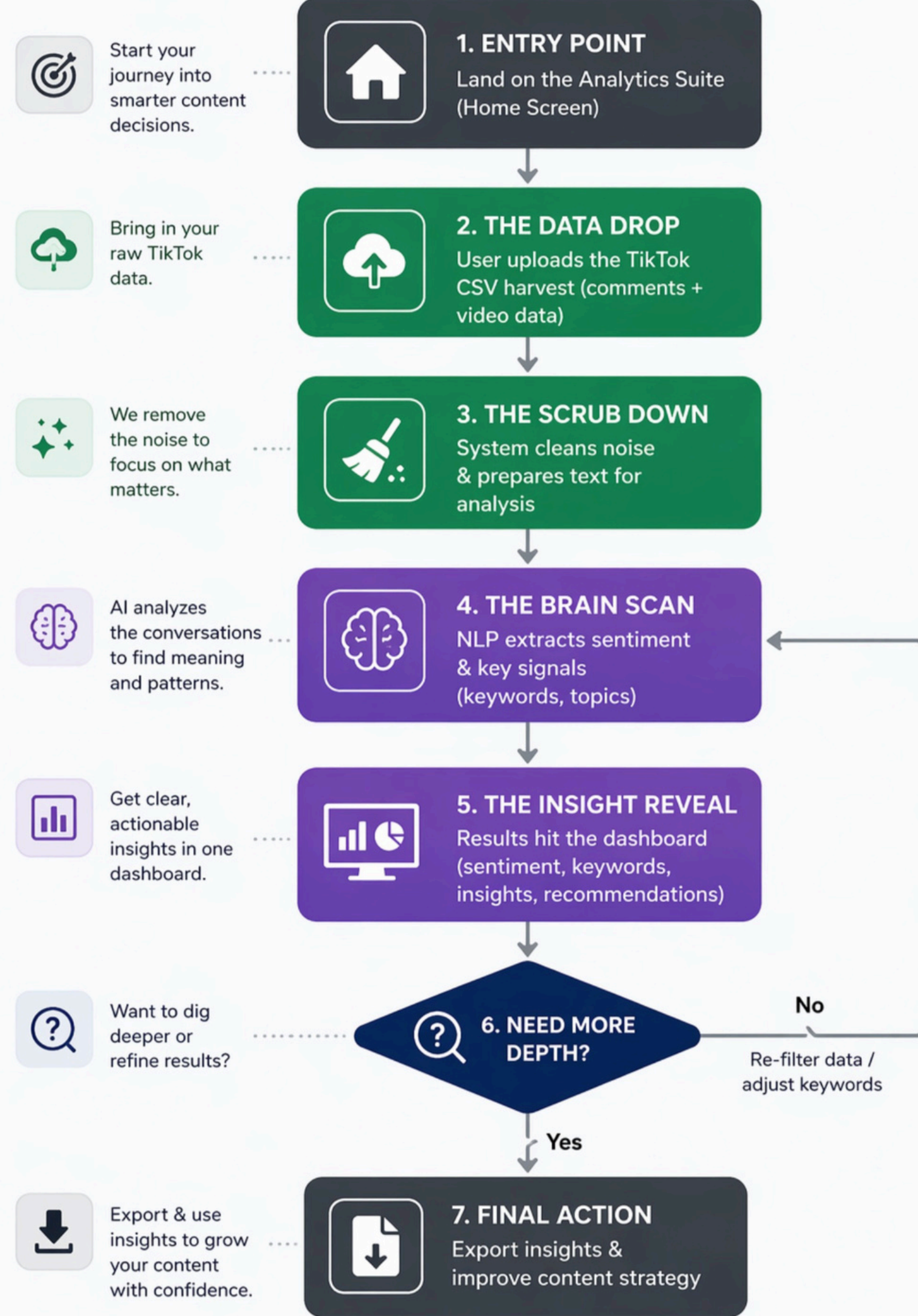
User Experience



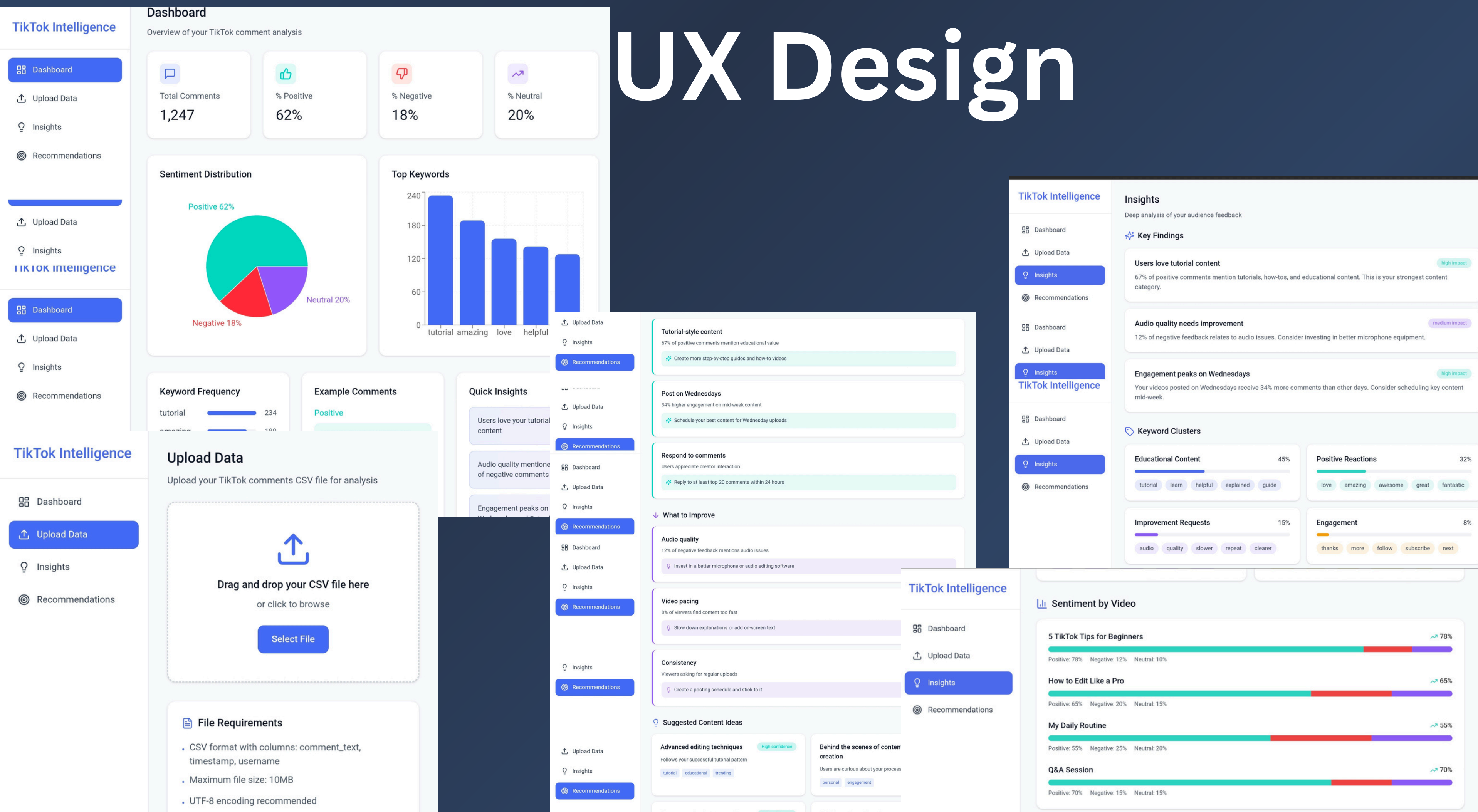
Moodboard



User Flow

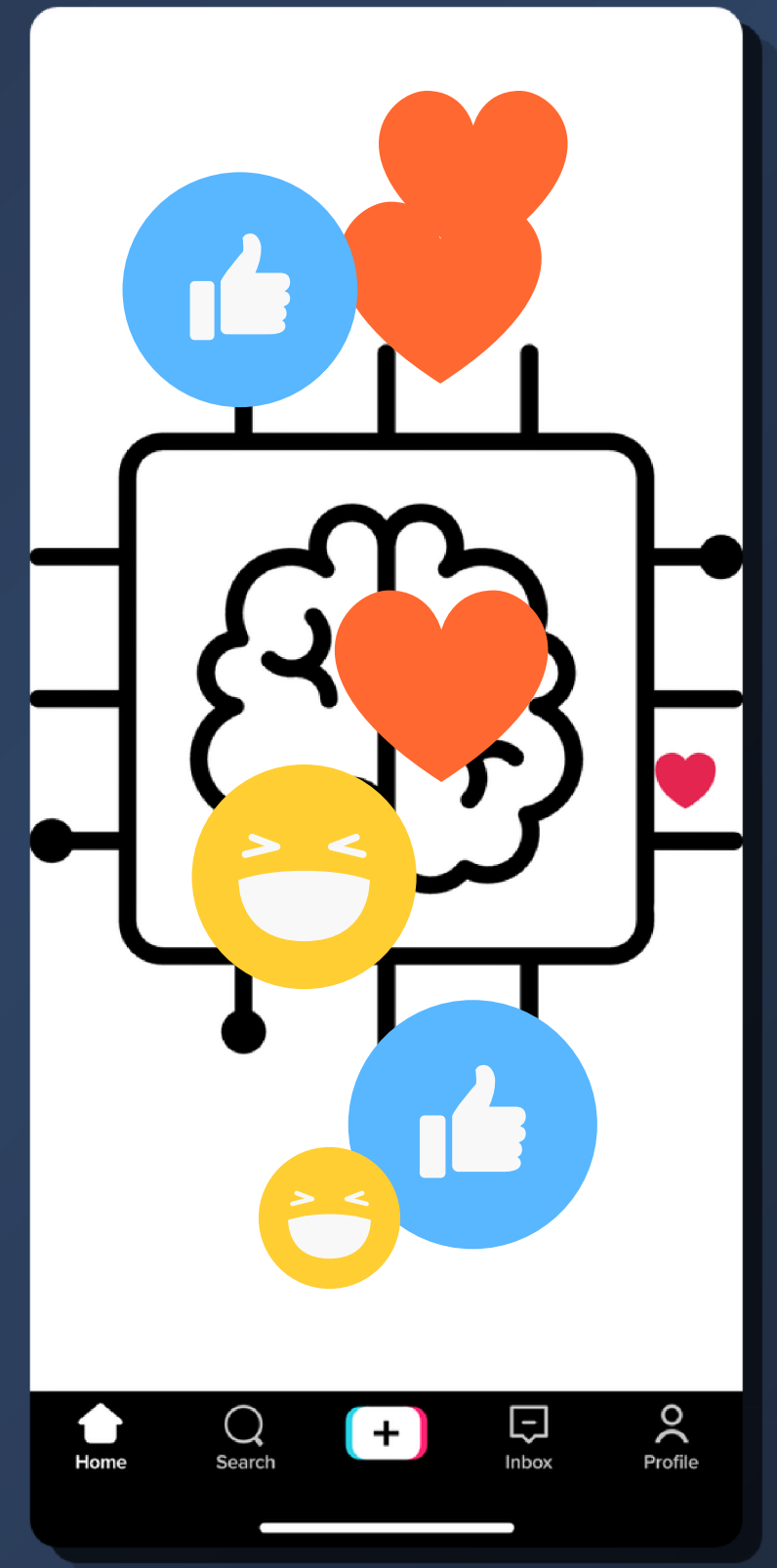


UX Design



Deliverable 4

Agile Plan



Task 3 Agile Board

Natural Language Processing

On track

Insights

Workflows

Task Tracker

Proj Roadmap

MoSCoW

Sprint 1

New view

Filter by keyword or by field

Backlog9

...

+

tiktok-creator-intelligence #27

As a creator, I want to view results in a dashboard so that insights are easy to understand.

Must Have

tiktok-creator-intelligence #28

As a creator, I want charts for sentiment distribution so that insights are visualized clearly.

Should Have

tiktok-creator-intelligence #29

As a creator, I want to compare videos so that I can identify better-performing content.

Should Have

tiktok-creator-intelligence #32

As a creator, I want dark mode so that the dashboard feels modern.

Could Have

tiktok-creator-intelligence #33

As a creator, I want to export results as PDF so that I can save insights.

Could Have

ToDo13

...

+

This item hasn't been started

tiktok-creator-intelligence #22

As a creator, I want comments classified as positive, negative, or neutral so that I can understand audience sentiment.

Must Have

tiktok-creator-intelligence #30

As a creator, I want upload instructions so that the app is easy to use.

Should Have

tiktok-creator-intelligence #31

As a creator, I want loading indicators so that I know processing is happening.

Should Have

tiktok-creator-intelligence #7

[S8] Connect end-to-end pipeline + Streamlit app

Phase 2: Building the SystemJun 5, 2026

tiktok-creator-intelligence #8

[S9] Evaluate system quality & test results

Phase 3: Polish & SubmitJun 12, 2026

tiktok-creator-intelligence #9

In Progress2

...

+

This is actively being worked on

tiktok-creator-intelligence #5

[S6] Prepare and clean TikTok dataset

Phase 2: Building the SystemMay 15, 2026

tiktok-creator-intelligence #6

[S7] Build isolated NLP modules (sentiment, keywords)

#38Phase 2: Building the SystemMay 22, 2026Cancelled

In Review1

...

+

tiktok-creator-intelligence #4

[S5] Set up project structure & agile workflow

Phase 1: Research & PlanningMay 8, 2026

+ Add item

Done3

...

+

This has been completed

tiktok-creator-intelligence #2

[S3] Research related work (SOTA — sentiment, NLP tools)

#14Phase 1: Research & PlanningApr 17, 2026Complete

tiktok-creator-intelligence #1

[S2] Define problem statement & system relevance

Phase 1: Research & PlanningApr 10, 2026

tiktok-creator-intelligence #3

[S4] Design UI wireframes (Streamlit layout)

Phase 1: Research & PlanningApr 24, 2026

08/05/2026

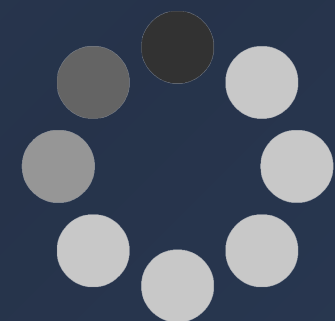
Task 3 MoSCoW View

The image displays a MoSCoW View Kanban board with four columns representing different priority levels. Each column contains a list of tasks, each with a unique ID, a description, and a 'Backlog' button.

- Must Have (7 items):**
 - tiktok-creator-intelligence #20: As a creator, I want to upload a CSV file so that I can analyze TikTok comments. [Backlog](#)
 - tiktok-creator-intelligence #21: As a creator, I want the system to clean comment text so that the analysis is more accurate. [Backlog](#)
 - tiktok-creator-intelligence #22: As a creator, I want comments classified as positive, negative, or neutral so that I can understand audience sentiment. [Backlog](#)
 - tiktok-creator-intelligence #23: As a creator, I want to view sentiment statistics so that I can quickly understand audience reactions. [Backlog](#)
 - tiktok-creator-intelligence #24: As a creator, I want the system to extract keywords so that I can identify common discussion topics. [Backlog](#)
- Should Have (4 items):**
 - tiktok-creator-intelligence #28: As a creator, I want charts for sentiment distribution so that insights are visualized clearly. [Backlog](#)
 - tiktok-creator-intelligence #29: As a creator, I want to compare videos so that I can identify better-performing content. [Backlog](#)
 - tiktok-creator-intelligence #30: As a creator, I want upload instructions so that the app is easy to use. [Backlog](#)
 - tiktok-creator-intelligence #31: As a creator, I want loading indicators so that I know processing is happening. [Backlog](#)
- Could Have (3 items):**
 - tiktok-creator-intelligence #32: As a creator, I want dark mode so that the dashboard feels modern. [Backlog](#)
 - tiktok-creator-intelligence #33: As a creator, I want to export results as PDF so that I can save insights. [Backlog](#)
 - tiktok-creator-intelligence #34: As a creator, I want keyword word clouds so that topics are easier to explore. [Backlog](#)
- Won't Have (3 items):**
 - tiktok-creator-intelligence #35: As a creator, I want real-time TikTok API integration. [Backlog](#)
 - tiktok-creator-intelligence #36: As a creator, I want mobile app support. [Backlog](#)
 - tiktok-creator-intelligence #37: As a creator, I want multilingual support. [Backlog](#)

At the bottom of the 'Should Have' column, there is a '+ Add item' button.

Task 3 Sprint View



Sprint 14

May 08 - May 21

tiktok-creator-intelligence #20

As a creator, I want to upload a CSV file so that I can analyze TikTok comments.

ToDo

tiktok-creator-intelligence #21

As a creator, I want the system to clean comment text so that the analysis is more accurate.

ToDo

tiktok-creator-intelligence #30

As a creator, I want upload instructions so that the app is easy to use.

ToDo

tiktok-creator-intelligence #31

As a creator, I want loading indicators so that I know processing is happening.

ToDo

Sprint 20Current

May 22 - Jun 04

Sprint 30

Jun 05 - Jun 18

+ Add Item

As a creator, I want the system to extract keywords so that I can identify common discussion topics. #24

Open Chinel0/tiktok-creator-intelligence Private



Chinel0 opened 2 weeks ago

Last edited by Chinel0

Acceptance Criteria:

System identifies the most frequently used words (topics).

System filters out "stop words" (e.g., "and", "the", "a").

A list of the top 5-10 keywords is displayed to the user.

Create sub-issue



Chinel0 moved this to Backlog in Natural Language Processing 2 weeks ago

Chinel0 added this to Natural Language Processing 2 weeks ago

Chinel0 moved this from Backlog to ToDo in Natural Language Processing 2 weeks ago

Assignees

No one - [Assign yourself](#)

Assign to Agent

Labels

No labels

Projects

Acceptance Criteria

As a creator, I want to upload a CSV file so that I can analyze TikTok comments. #20

Open Chinel0/tiktok-creator-intelligence Private



Chinel0 opened 2 weeks ago

Last edited by Chinel0

Acceptance Criteria:

User can select a .csv file from their computer.

The system checks if the file format is correct.

Data from the CSV appears in the app after upload.

Create sub-issue



Chinel0 moved this to Backlog in Natural Language Processing 2 weeks ago

Chinel0 added this to Natural Language Processing 2 weeks ago

Chinel0 moved this from Backlog to ToDo in Natural Language Processing 2 weeks ago

Assignees

No one - [Assign yourself](#)

Assign to Agent

Labels

No labels

Projects

Natural Language Processing

Status

ToDo

dates

No date

MoSCoW

Must Have

Story Points

3

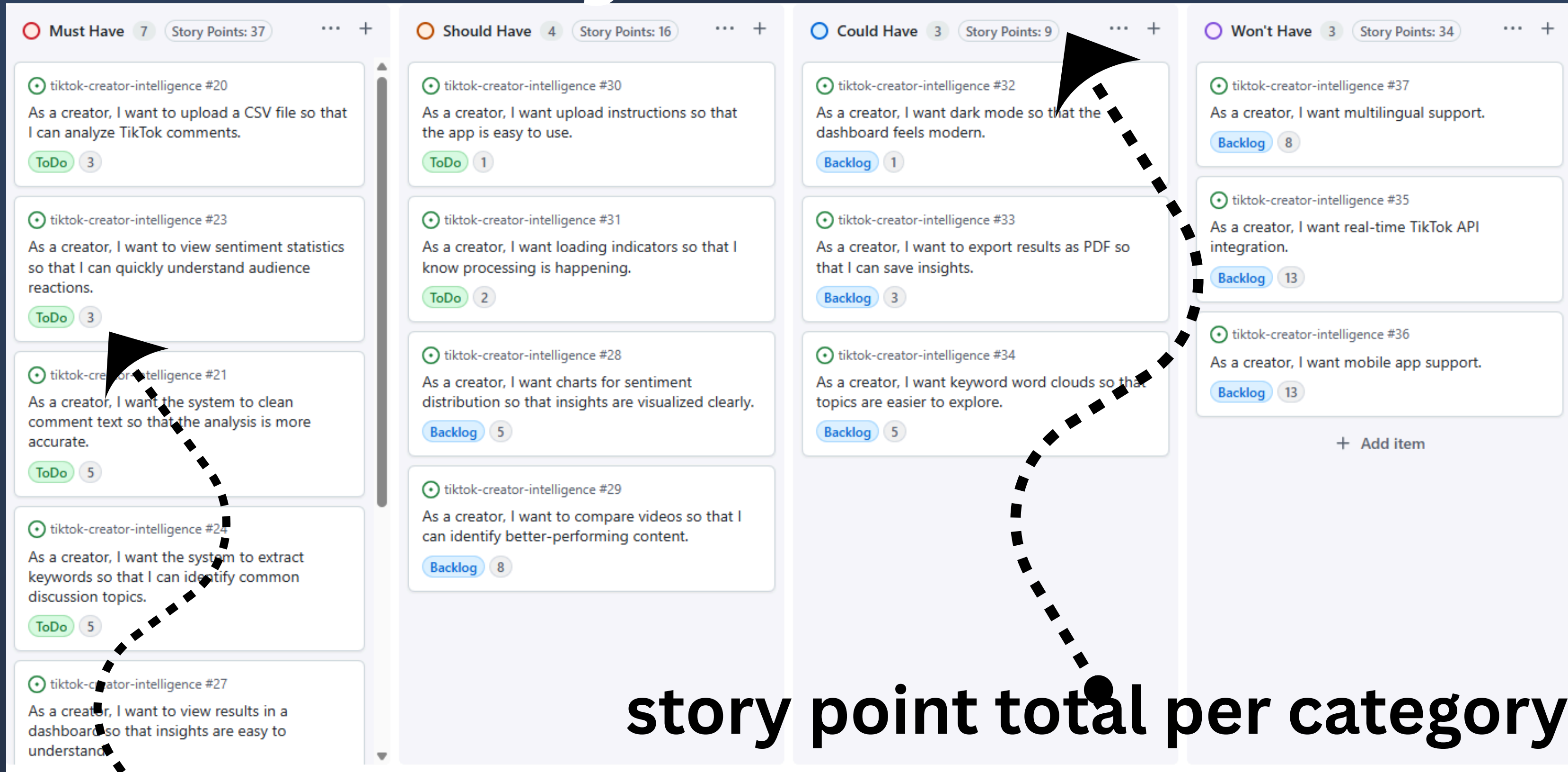
Sprint

Sprint 1 •
May 8 - May 21

Task 3

08/05/2026

Task 3 Story Points

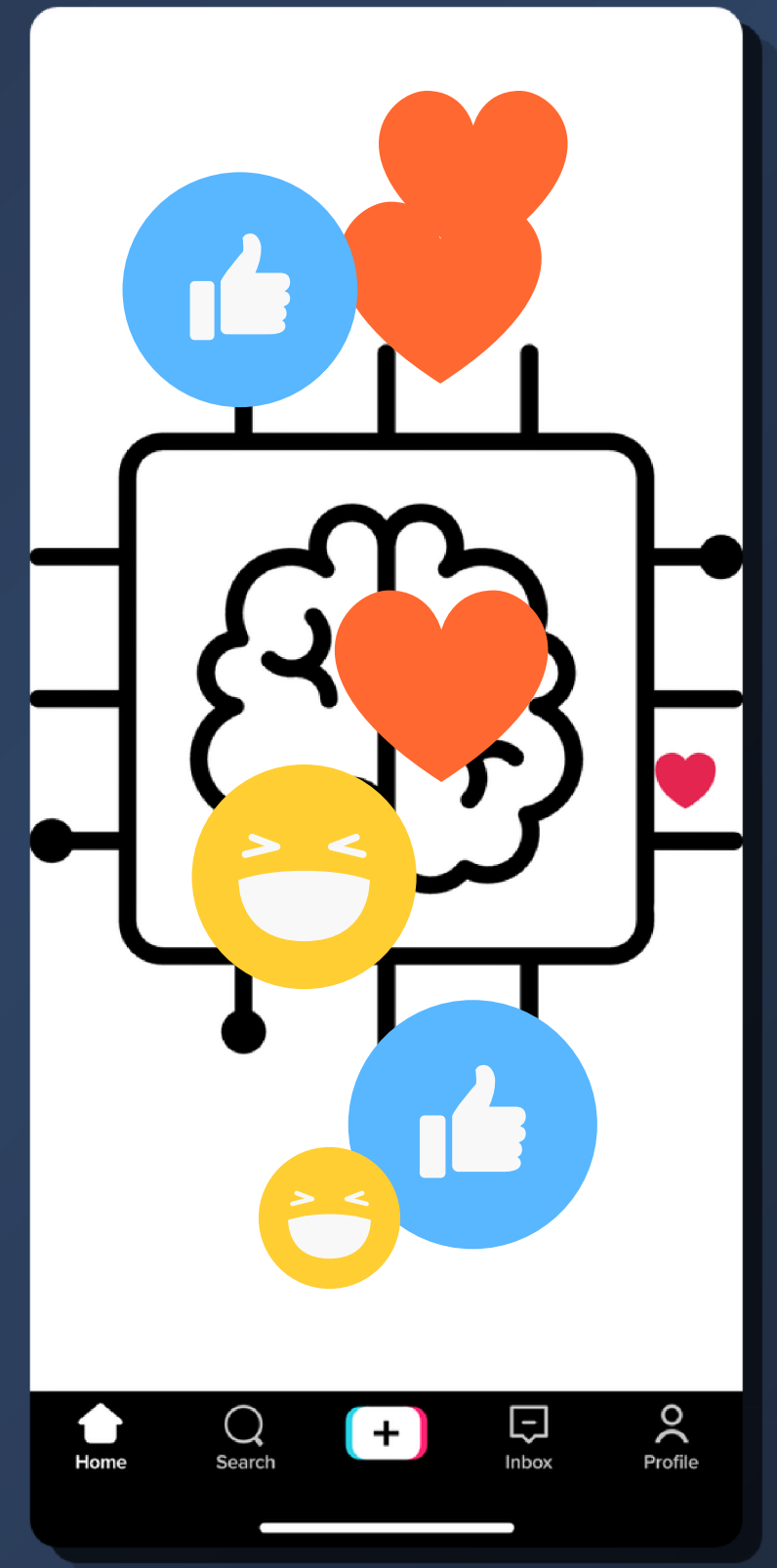


story point total per category

Story point per task

Deliverable 5

Data Strategy



Data Source

Primary Data Source

Personal TikTok account — @ichbinnelo (public profile)

Retrieval Method

Custom Python scraper using Playwright browser automation. Intercepts TikTok's internal comments API (/api/comment/list/) directly from the network — no third-party library.

Data Scope

- 161 videos
- 1,211 comments
- 9 content categories
- scraped May 2026

15/05/2026

Data Lineage & Legal Check

 **Copyright**

my own account no third party rights

 **Terms of Use**

Personal data collection for
educational research

 **GDPR**

Public comments, no PII stored,
university project only

 **Scientific**

Full citation: Ariyus, D., Manongga, D., & Sembiring, I. (2024). Enhancing Sentiment Analysis of Indonesian Tourism Video Content Commentary on TikTok: A FastText and Bi-LSTM Approach. Engineering, Technology & Applied Science Research, 14(6), 18020–18028.

Pre-processing

```
import re

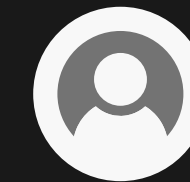
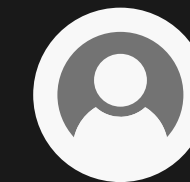
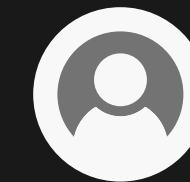
def clean_text(text):
    if not isinstance(text, str): return ""
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#\w+', '', text)
    text = re.sub(r'^\w\s', '', text)
    text = re.sub(r'\d+', '', text)
    text = re.sub(r'^\x00-\x7F+', '', text)
    text = re.sub(r'\s+', ' ', text)
    tokens = [t for t in text.split() if len(t) >= 3]
    return ' '.join(tokens)

examples = [
    "omg I love this so much!!! 🤩🤩 #foodtok #fyp check https://t.co/abc123",
    "@ichbinnelo please do more cooking videos!! 🍳",
    "this and starting my seedlings #CapCut #foodtok",
    "I wish you a wonderful day ❤️",
    "berlin is nice for tourists but not for residents",
]

print(f'{"RAW INPUT":<55} → CLEANED OUTPUT')
print('-' * 95)
for raw in examples:
    cleaned = clean_text(raw)
    print(f'{raw[:55]:<55} → {cleaned if cleaned else "(empty after cleaning)"}')
```

Python

RAW INPUT	→	CLEANED OUTPUT
omg I love this so much!!! 🤩🤩 #foodtok #fyp check https	→	omg love this much check
@ichbinnelo please do more cooking videos!! 🍳	→	please more cooking videos
this and starting my seedlings #CapCut #foodtok	→	this and starting seedlings
I wish you a wonderful day ❤️	→	wish you wonderful day
berlin is nice for tourists but not for residents	→	berlin nice for tourists but not for residents



15/05/2026

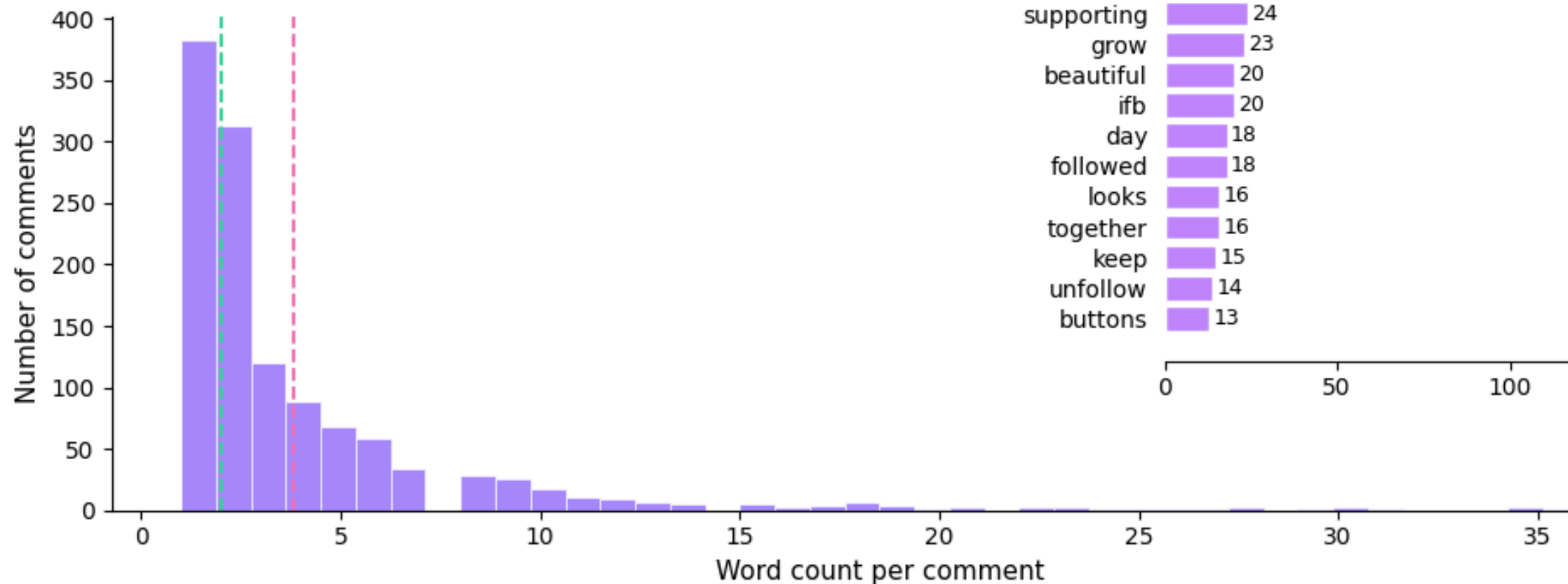
Exploratory Showcase

Total comments: 1,211

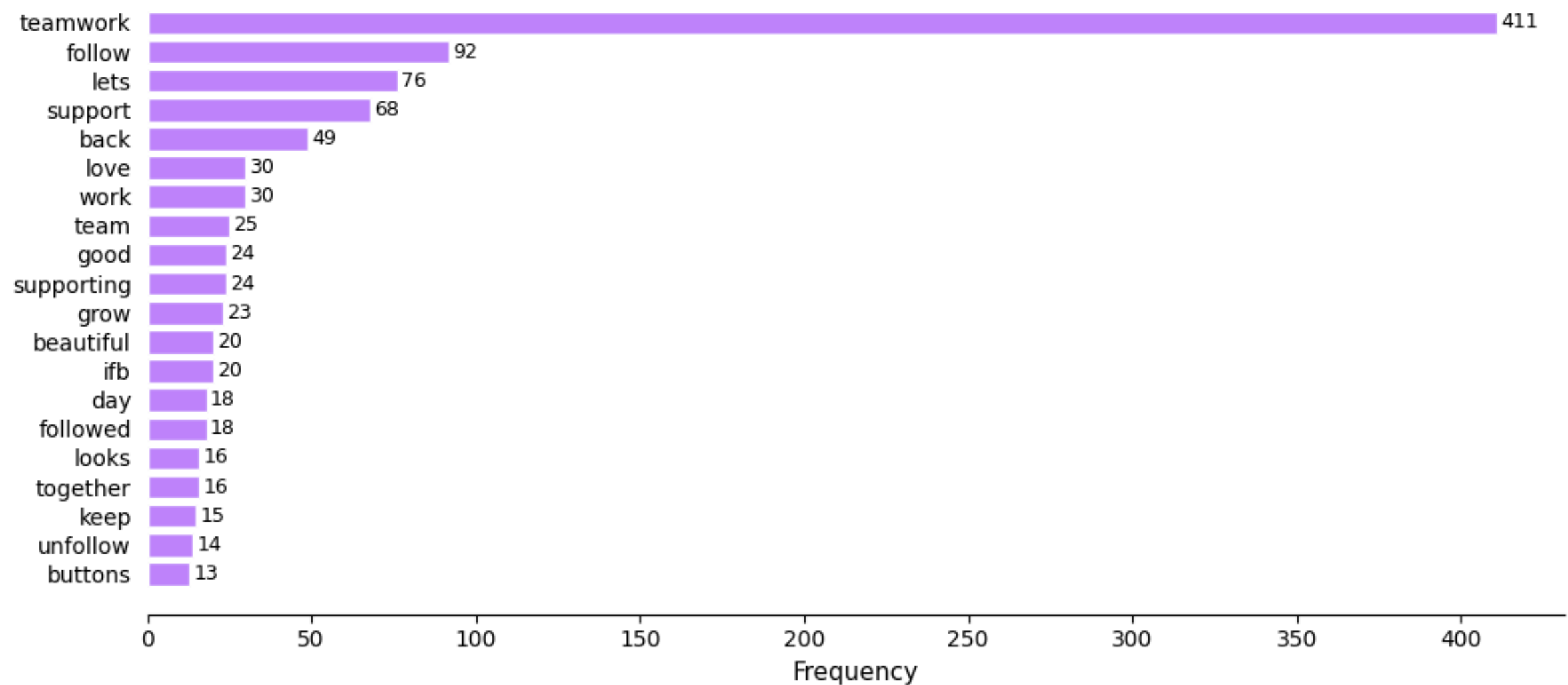
English comments: 944 (78%) — remaining 267 were auto-translated from German and other languages

Average comment length: 3.8 words — TikTok comments are very short, which is why VADER was chosen over longer-context models

Comment Length Distribution



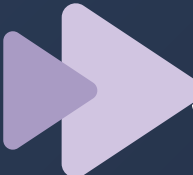
Top 20 Most Frequent Terms in Comments (@ichbinnelo)



15/05/2026

Applicability



-  SIGNAL → comment_text (what the audience said)
-  SIGNAL → like_count (how many agreed with the comment)
-  SIGNAL → comment_date (spot trends over time)
-  SIGNAL → video_id (links comment to content category)

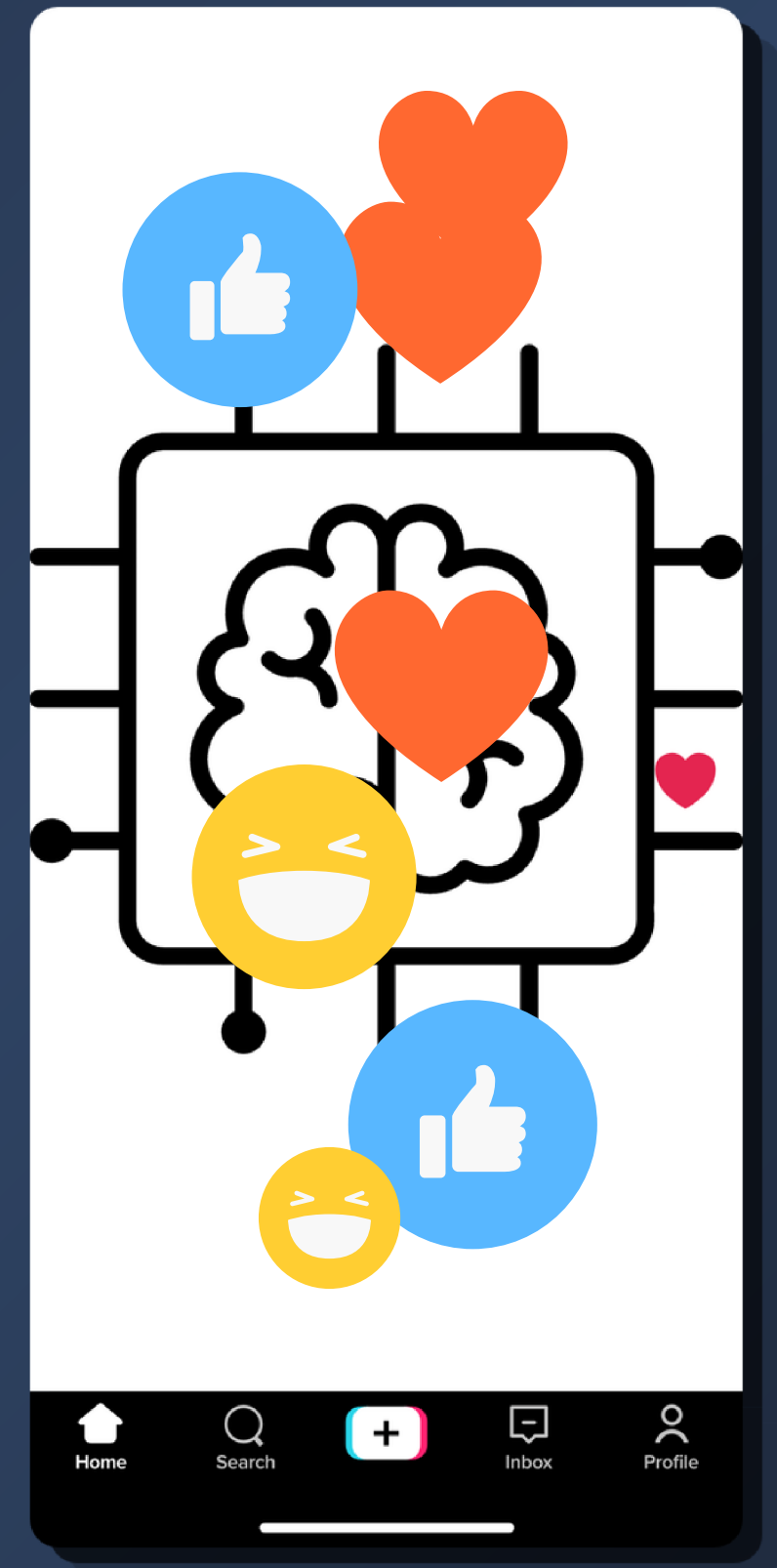
NOISE → scraped_at (internal timestamp, not analytical)

NOISE → original_text (pre-translation backup, not used)

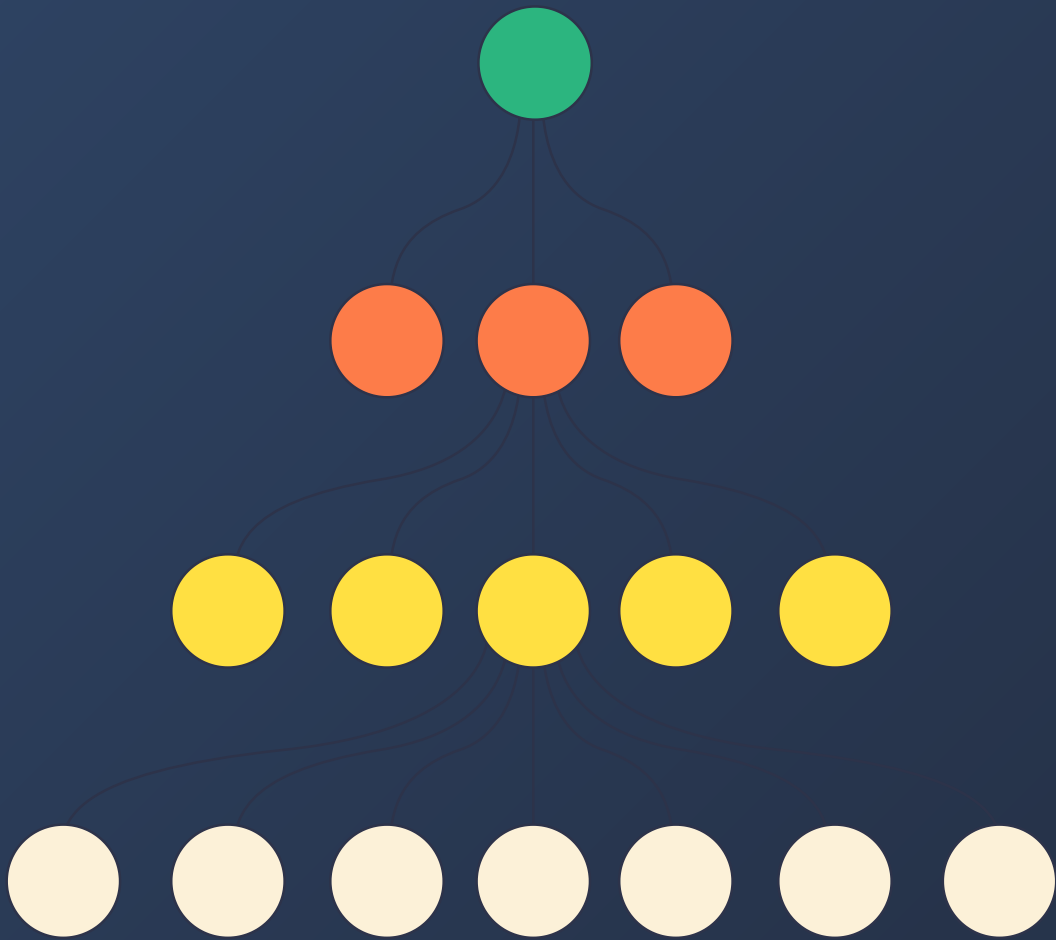
NOISE → language (processing artifact, job done)

Deliverable 6

NLP Modeling



Method & Model Choice



Method 1 — Sentiment Classification

Classify each comment as Positive, Negative, or Neutral based on the words used.

Model: VADER — vaderSentiment v3.3.2 (Hutto & Gilbert, 2014)

Why: Built specifically for short informal social media text. Handles slang, caps ("LOVE THIS"), exclamation marks, and emojis. Free, offline, no training data needed. Scores 1,211 comments in under 1 second.

Method 2 — Keyword Extraction

Find the most important words per content category to show what each audience talks about.

Model: TF-IDF — scikit-learn (unigrams + bigrams, English stopwords removed)

Why: Identifies words that are common in one category but rare in others — exactly the "what do food viewers say that lifestyle viewers don't?" question. Fully interpretable, no GPU, no training data.



Technical Setup

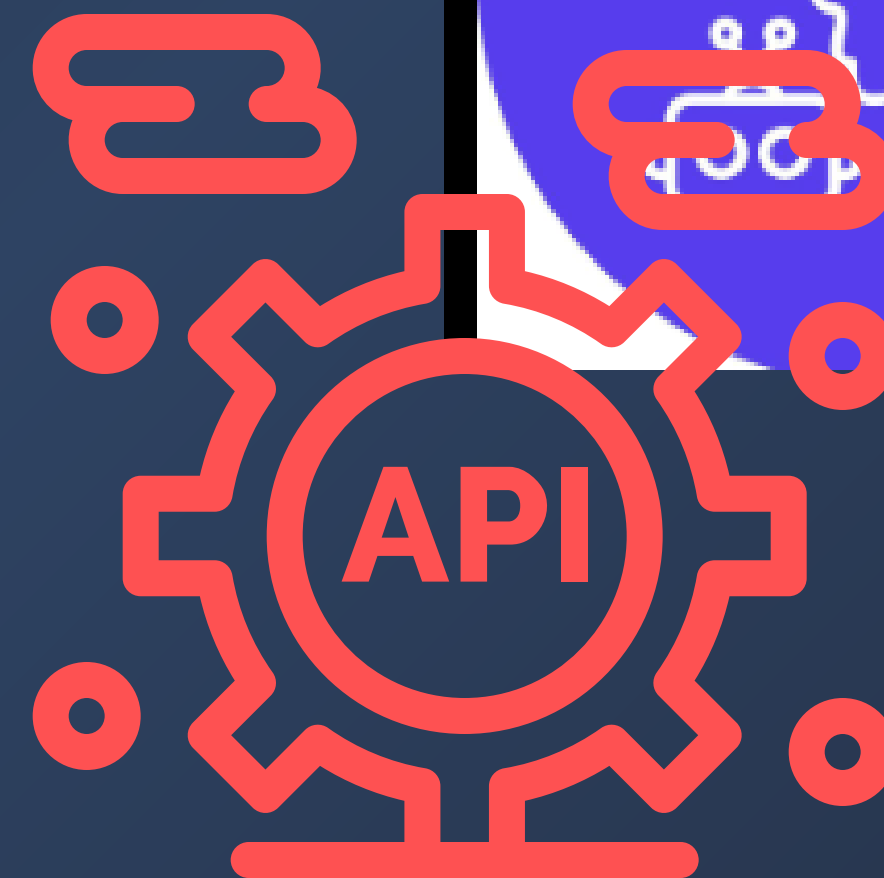
Environment: Local machine · Windows 11 · Python 3.12 · Jupyter Notebook

Library Role

- vaderSentiment Sentiment scoring
- scikit-learn TF-IDF keyword extraction
- pandas Data loading and filtering
- nltk Stopword list
- deep-translator Pre-processing: German → English

Access: No API key. No account. No internet after install. Fully private, comment data never leaves the machine.

Cost & Latency: £0.00 · Full pipeline runs in under 5 seconds



Minimal Working Example

```
# — Minimal Working Example: Sentiment Classification —————
# Input: raw comment text → Output: compound score + label
# Model: VADER (vaderSentiment, Hutto & Gilbert 2014)

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
import pandas as pd

analyzer = SentimentIntensityAnalyzer()

def score(text):
    c = analyzer.polarity_scores(str(text))['compound']
    label = 'POSITIVE' if c >= 0.05 else ('NEGATIVE' if c <= -0.05 else 'NEUTRAL')
    return round(c, 3), label

# Run on 5 real comments from the dataset
comments = pd.read_csv('data/comments_20260526_233400.csv', encoding='utf-8-sig')
sample = comments['comment_text'].dropna().sample(5, random_state=42).tolist()

print(f'{"COMMENT":<50} {"SCORE":>7} LABEL')
print('-' * 75)
for text in sample:
    compound, label = score(text)
    print(f'{str(text)[:50]:<50} {compound:>+.3f} {label}')
```

COMMENT	SCORE	LABEL
Wow yummy 🍴 with my favorite song 🎵	+0.881	POSITIVE
let's get this started 👍	+0.000	NEUTRAL
Another day another lets go lets roll 🤗👍🌟	+0.778	POSITIVE
nice routinee	+0.421	POSITIVE
Won't unfollow	+0.000	NEUTRAL

```

videos = pd.read_csv('data/videos_merged.csv', encoding='utf-8-sig')

# Join so each comment inherits its video's category
merged = comments.merge(videos[['video_id', 'video_type']], on='video_id', how='left')

def clean(text):
    text = re.sub(r'^a-zA-Z\s', ' ', str(text).lower())
    return re.sub(r'\s+', ' ', text).strip()

# Extract keywords for one category – change this to explore others
CATEGORY = 'Food & Cooking'
corpus = merged[merged['video_type'] == CATEGORY]['comment_text'].dropna().apply(clean).tolist()

vectorizer = TfidfVectorizer(max_features=200, stop_words='english',
                             ngram_range=(1, 2), min_df=2)
vectorizer.fit(corpus)

scores = np.array(vectorizer.transform(corpus).mean(axis=0)).flatten()
top_idx = scores.argsort()[::-1][:10]
keywords = [(vectorizer.get_feature_names_out()[i], round(scores[i], 4)) for i in top_idx]

print(f'Top 10 keywords – {CATEGORY}')
print('-' * 35)
for word, score in keywords:
    bar = '█' * int(score * 300)
    print(f' {word:<22} {score:.4f} {bar}')
```

Top 10 keywords – Food & Cooking

yummy	0.0966	
looks	0.0917	
looks yummy	0.0455	
thanks	0.0416	
think	0.0416	
just	0.0416	
love	0.0387	

Sample Output & Observations

COMMENT	SCORE	LABEL

omg I love this so much it looks amazing	+0.840	POSITIVE
this is my favorite type of content please keep making	+0.649	POSITIVE
I wish you a wonderful day	+0.751	POSITIVE
berlin is nice for tourists but not for residents	+0.226	POSITIVE
what was the first thing in the video this one with str	+0.572	POSITIVE
it is not such a bad winter day is it	+0.431	POSITIVE
please do it because I am not that famous yet	+0.318	POSITIVE
I have been trying to pay off my debt for years this re	-0.440	NEGATIVE
I add back everyone who adds me	+0.000	NEUTRAL
My condolences	+0.000	NEUTRAL

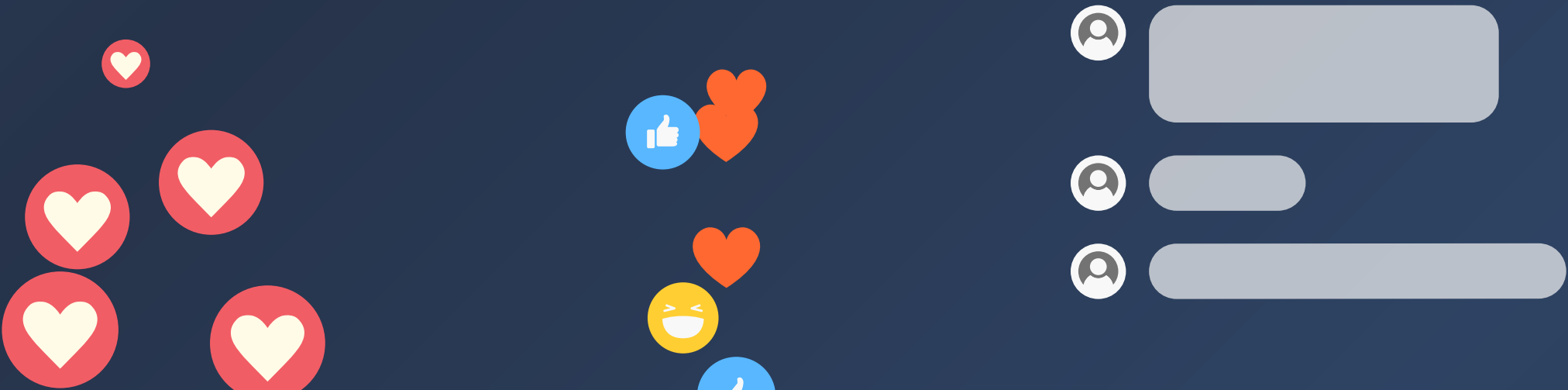
Full corpus sentiment distribution:

POSITIVE	723	(59.7%)	#####
NEUTRAL	433	(35.8%)	#####
NEGATIVE	55	(4.5%)	#



Observations:

- Genuine compliments and encouragement are scored confidently and correctly
- Negative sentiment is rare in the corpus — expected for a creator's own comment section
- Very short comments ("wow", "same", "done") score neutral — not enough words for VADER to read
- Engagement-bait comments ("teamwork lets go") score weakly positive despite carrying no real opinion about the content — the video_type filter removes these before analysis



Critical Scaling Points

1. VADER misses sarcasm

"sure, love spending all my money on debt 🙄" scores positive. Fine at 1,211 comments. At 50,000 it misleads the dashboard.

Mitigation: Flag scores between -0.2 and +0.2 as uncertain rather than neutral.

2. TF-IDF breaks on small categories

Books & Reading has only 4 videos — too few documents for reliable keywords.

Mitigation: Suppress keyword output for categories with fewer than 10 videos.

3. Translation rate limits

deep-translator (Google Translate) has undocumented free-tier limits. At 10,000+ comments it would fail mid-run.

Mitigation: Batch in groups of 100 with a short delay, or switch to offline Helsinki-NLP/opus-mt.

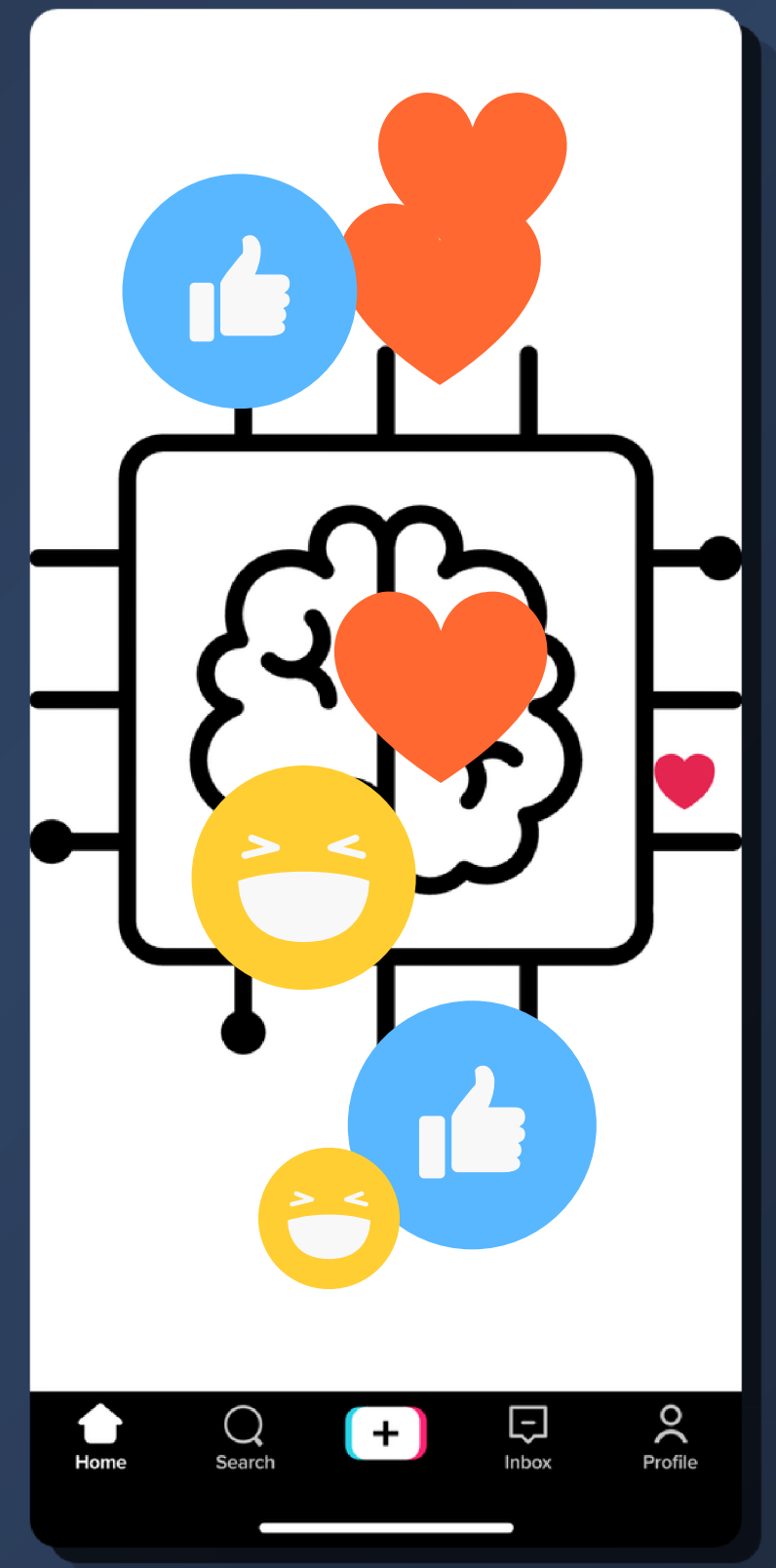
4. Video categories don't generalise

The hashtag rules were written for @ichbinnelo. A different creator would get mostly Unknown.

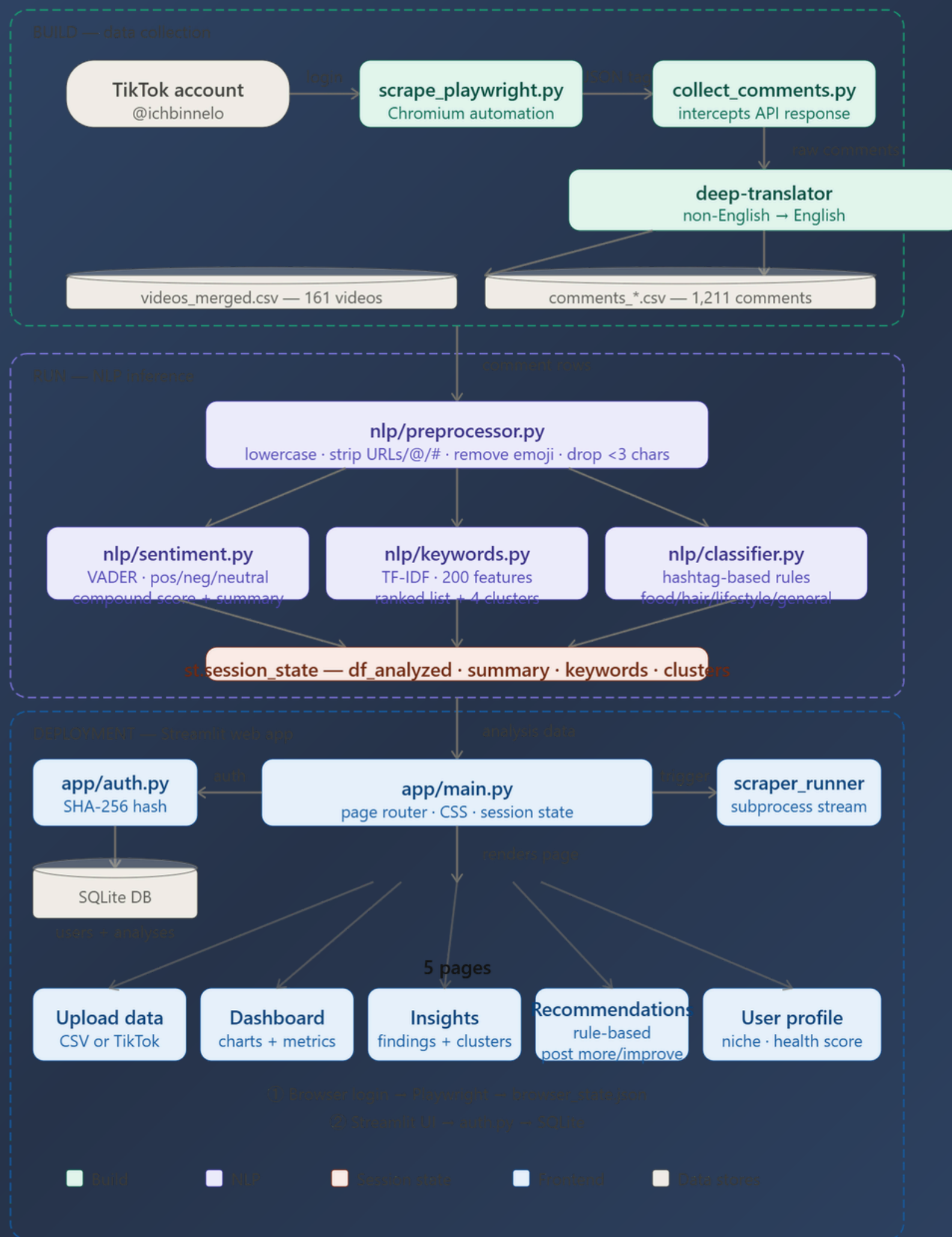
Mitigation: Make category rules a user-configurable file in the app.

Deliverable 7

End-2-End-Systems



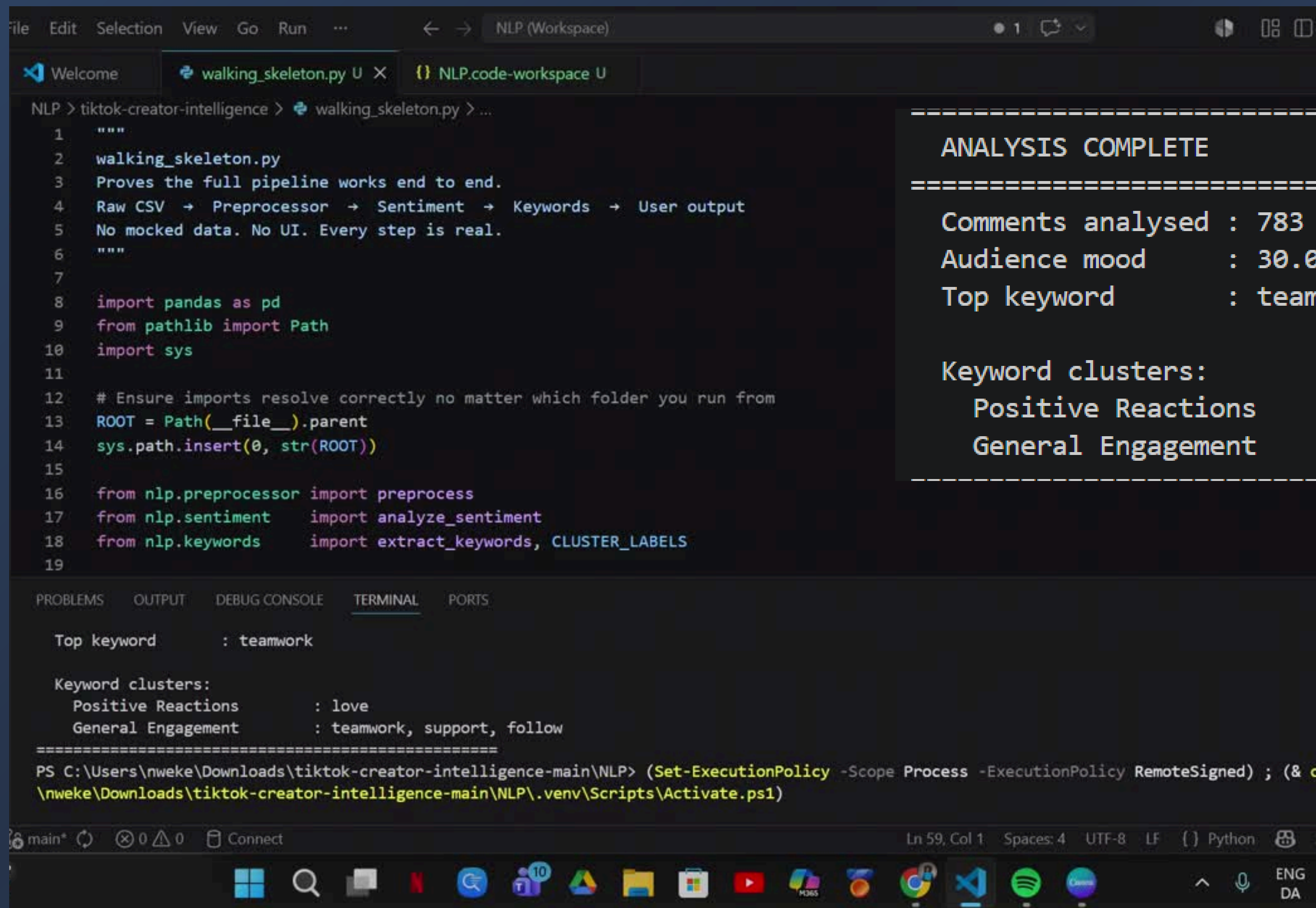
System Architecture Diagram



Pipeline Component Breakdown

Junction	Input Format	Tool	Output Format
Raw Data → Preprocessor	Excel file (.xlsx), 1211 rows, 15 columns	pandas + preprocessor.py	Cleaned DataFrame, 944 English rows, new clean_comment column
Preprocessor → Sentiment	Cleaned DataFrame with clean_comment column	VADER (nltk) via sentiment.py	DataFrame + new sentiment column (positive/negative/neutral) + score
Preprocessor → Keywords	Cleaned DataFrame with clean_comment column	TF-IDF (scikit-learn) via keywords.py	List of top 20 keywords + 4 keyword clusters with percentages
NLP Modules → Insights	Sentiment labels + keyword clusters	insights.py	Dictionary of findings and recommendations
Insights → UI	Python dictionaries and DataFrames	Streamlit via app/main.py	Visual dashboard with charts, cards, recommendations
UI → Database	User profile + analysis results	SQLite via sqlite3	Stored analysis history for returning users

Walking Skeleton Demo



The image shows a VS Code editor window with a Python script named `walking_skeleton.py` open. The script is a walking skeleton for an NLP pipeline. The terminal at the bottom shows the output of the script, which includes the top keyword and keyword clusters.

```
File Edit Selection View Go Run ... NLP (Workspace) 1
```

```
1 """
2 walking_skeleton.py
3 Proves the full pipeline works end to end.
4 Raw CSV → Preprocessor → Sentiment → Keywords → User output
5 No mocked data. No UI. Every step is real.
6 """
7
8 import pandas as pd
9 from pathlib import Path
10 import sys
11
12 # Ensure imports resolve correctly no matter which folder you run from
13 ROOT = Path(__file__).parent
14 sys.path.insert(0, str(ROOT))
15
16 from nlp.preprocessor import preprocess
17 from nlp.sentiment import analyze_sentiment
18 from nlp.keywords import extract_keywords, CLUSTER_LABELS
19
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

```
Top keyword      : teamwork

Keyword clusters:
  Positive Reactions      : love
  General Engagement      : teamwork, support, follow
=====
PS C:\Users\uweke\Downloads\tiktok-creator-intelligence-main\NLP> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c
\nweke\Downloads\tiktok-creator-intelligence-main\NLP\.venv\Scripts\Activate.ps1)
```

main* 0 0 Connect Ln 59, Col 1 Spaces: 4 UTF-8 LF Python

ANALYSIS COMPLETE

Comments analysed : 783

Audience mood : 30.0% positive

Top keyword : teamwork

Keyword clusters:

Positive Reactions : love

General Engagement : teamwork, support, follow

Bottleneck Analysis

Bottleneck 1

35% of comments silently dropped 1,211 comments loaded → only 783 analysed. The preprocessor discards translated comments because their language column still says 'de' or 'fr' even though the text is already English.

Bottleneck 2

Keyword output polluted by engagement spam Top keyword is teamwork (442x). This comes from follow-train posts, not real content feedback. TF-IDF runs on all comments at once



Next 3 Task

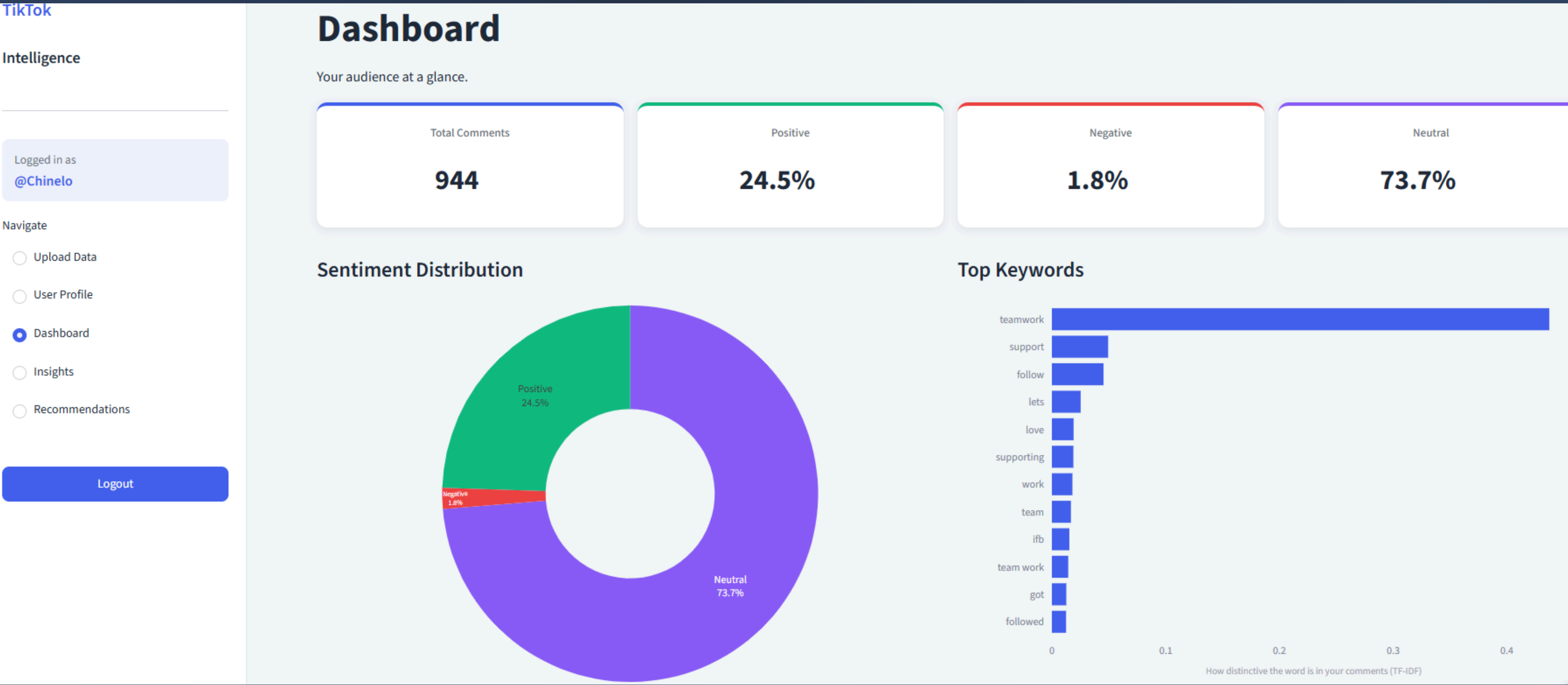
1. Fix the language filter — keep translated comments that have an `original_text` value. Recovers the missing 428 comments.
2. Filter out Engagement / Growth videos before keyword extraction. Removes the teamwork/follow spam and surfaces real content keywords.
3. Connect pipeline output to the Streamlit dashboard — replace `print()` statements with live UI output. This completes the bridge from backend to prototype.



What's Under the Hood?

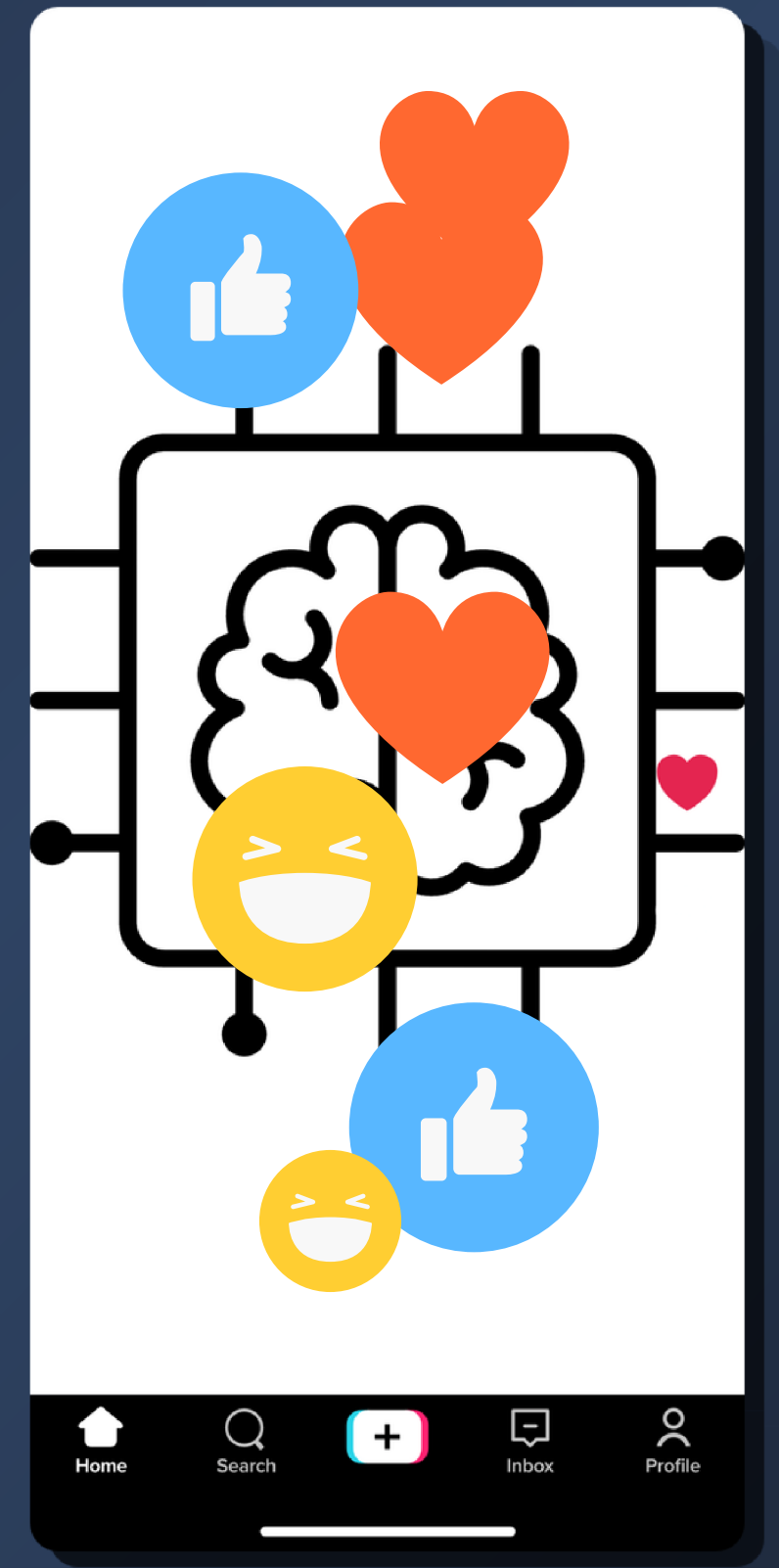
NLP Model Used
VADER: open-source sentiment lexicon

TF-IDF: a statistical method, plus rule-based analytics



Deliverable 8

Evaluation & Quality



Performance Metrics

Metric	Score	Justification
Accuracy	91.90%	Overall correctness; test set mirrors the real class balance
Macro F1	94.10%	Weighs all 3 classes equally — the rare negative class matters most to creators
Negative-class F1	100%	The high-value class: criticism detection (2/2 caught)

Gain over naive baseline:

+37.8 pts accuracy

+70.7 pts Macro F1



Key Finding from Evaluation

Evaluated on 37 real comments from my own account, manually labeled (held-out test set). Preprocessing stripped the emojis and punctuation VADER uses as sentiment signals.

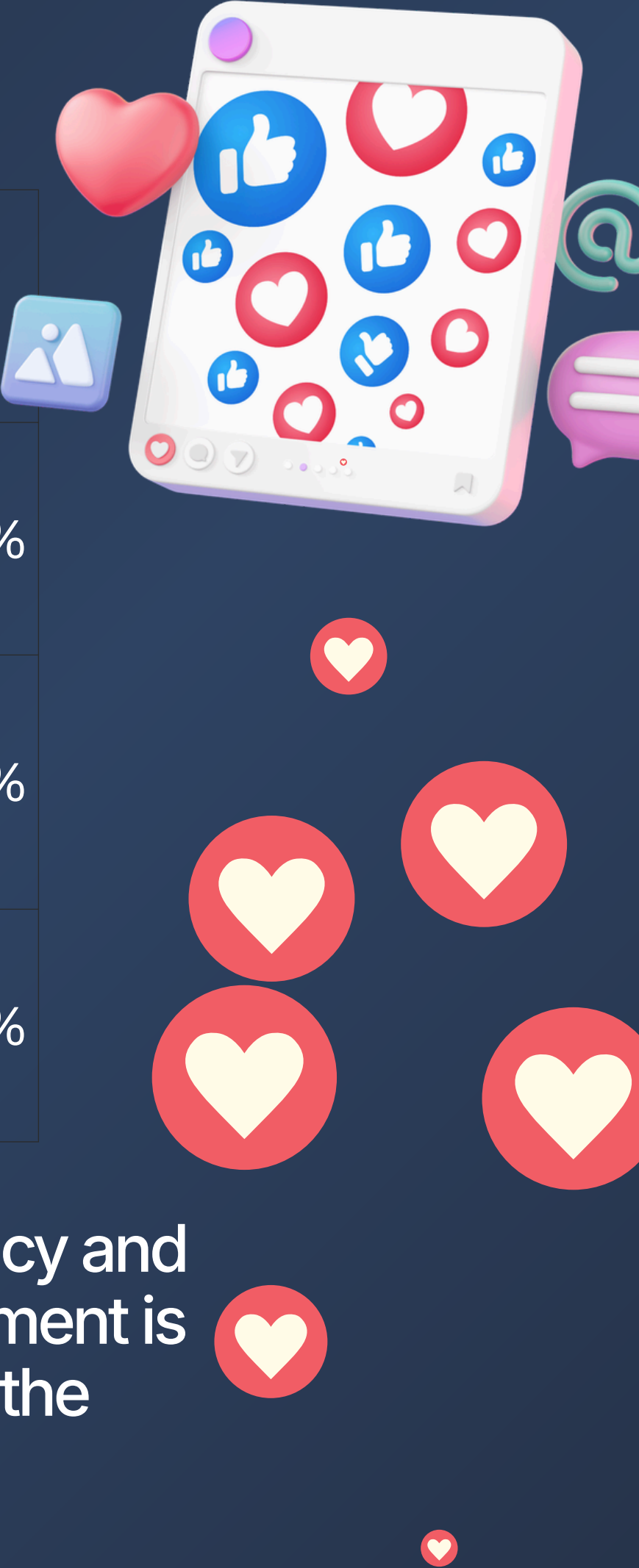
Fix applied: VADER now runs on the original comment text; cleaning is kept for TF-IDF keywords only.

Result: +5.4 pts accuracy, +9.8 pts Macro F1 from the emoji fix.

Benchmark Comparison

System	Accuracy	Macro F1
Baseline A: Majority class (always positive)	54.10%	23.40%
Baseline B: VADER on cleaned text (emojis stripped)	86.50%	84.30%
Full system: VADER emoji-aware (current)	91.90%	94.10%

Keeping emojis through to the sentiment analyser contributes +5.4 pts accuracy and +9.8 pts Macro F1 over the ablated version, confirming that emoji-aware sentiment is the highest-value design decision in the pipeline. On TikTok, the emoji often is the sentiment.



Pipeline Efficiency

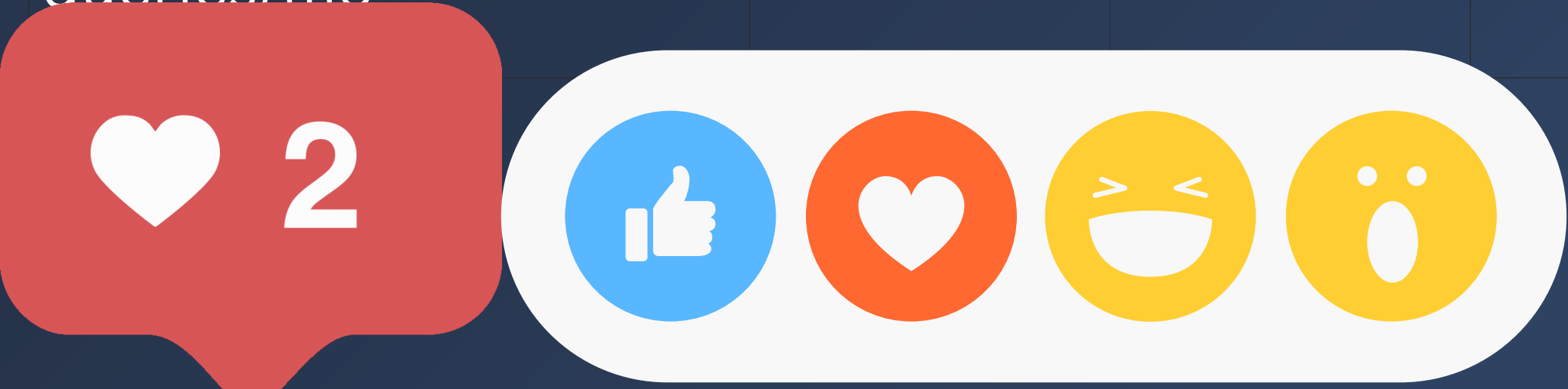
Scale	Avg Latency	Worst Case	Cost/Query	Est. Monthly
100 queries/mo	0.28 s	0.34 s	€0.00	€0.00
1,000 queries/mo	0.28 s	0.34 s	€0.00	€0.00
10,000 queries/mo	0.28 s	0.34 s	€0.00	€0.00

Bottleneck: sentiment scoring (VADER) 39% of total latency. Preprocessing is now only 9%.

Throughput: 4,300 comments/second end-to-end (1,211 comments + 161 videos in 0.28 s).

Cost: zero. Fully local pipeline, no API calls ,VADER + TF-IDF + statistics run on device.

Note: TikTok scraping (55 min for 161 videos) is a one-time data collection step, separate from the analysis pipeline.



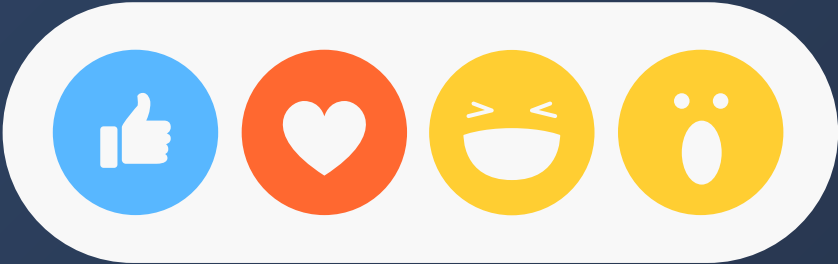
Error Analysis

Category	Count	% of data	Fix
Non-English comments excluded	267	22.00%	HIGH — trust short ASCII comments as English
language-detection false positives	142	11.70%	(1–2 word comments like "wow" misdetected)
Emoji-only comments	147	12.10%	Fixed — VADER now reads original text
Test-set misclassifications	3 of 37	8.10%	neutral/positive boundary, ± 0.05 threshold
Potential sarcasm	5	0.40%	Route to transformer model (future work)

[DONE] Emoji sentiment VADER now runs on original text (+5.4 pts accuracy, measured)

[NEXT] Language filter trust short ASCII comments as English instead of langdetect's guess; recovers ~142 comments at zero risk

[FUTURE] Slang & sarcasm custom VADER lexicon for hype slang ("hard", "crazy", "sick"), or swap to cardiffnlp/twitter-roberta-base-sentiment



SUS Results per Question (n = 6)

Combined SUS score: 87.5 / 100

#	Statement	Avg (1–5)	Good =
1	I think that I would like to use this system frequently.	4.3	high ✓
2	I found the system unnecessarily complex.	1.7	low ✓
3	I thought the system was easy to use.	4.7	high ✓
4	I think that I would need the support of a technical person to be able to use this system.	1.3	low ✓
5	I found the various functions in this system were well integrated.	4.3	high ✓
6	I thought there was too much inconsistency in this system.	2	low ✓
7	I would imagine that most people would learn to use this system very quickly.	4.5	high ✓
8	I found the system very cumbersome to use.	1.3	low ✓
9	I felt very confident using the system.	4.8	high ✓
10	I needed to learn a lot of things before I could get going with this system.	1.3	low ✓

User Evaluation

Was anything flat-out WRONG with the app? What?

5 responses

Its a very good idea but the categorization of comments is very unreliable.

I didn't notice anything wrong, it looks okay to me and it's something I could use

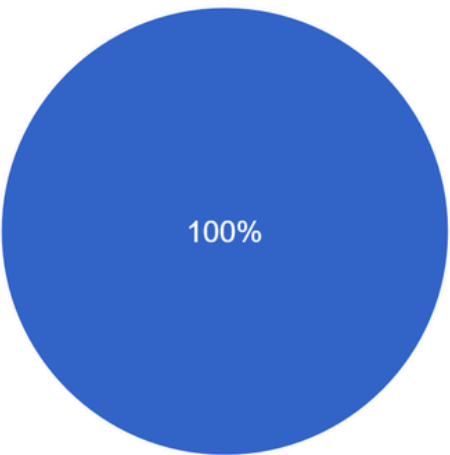
Some words like content and love content were flagged differently and that is confused as a user.

Slang being seen as negative

no because it tells me what to focus on when making contents for my audience, but I noticed some repetition of words, I feel like it still needs to be More specific and clear in breaking down the analysis of contents so as not to confuse new users.

Register + upload both files + run analysis, how did it go?

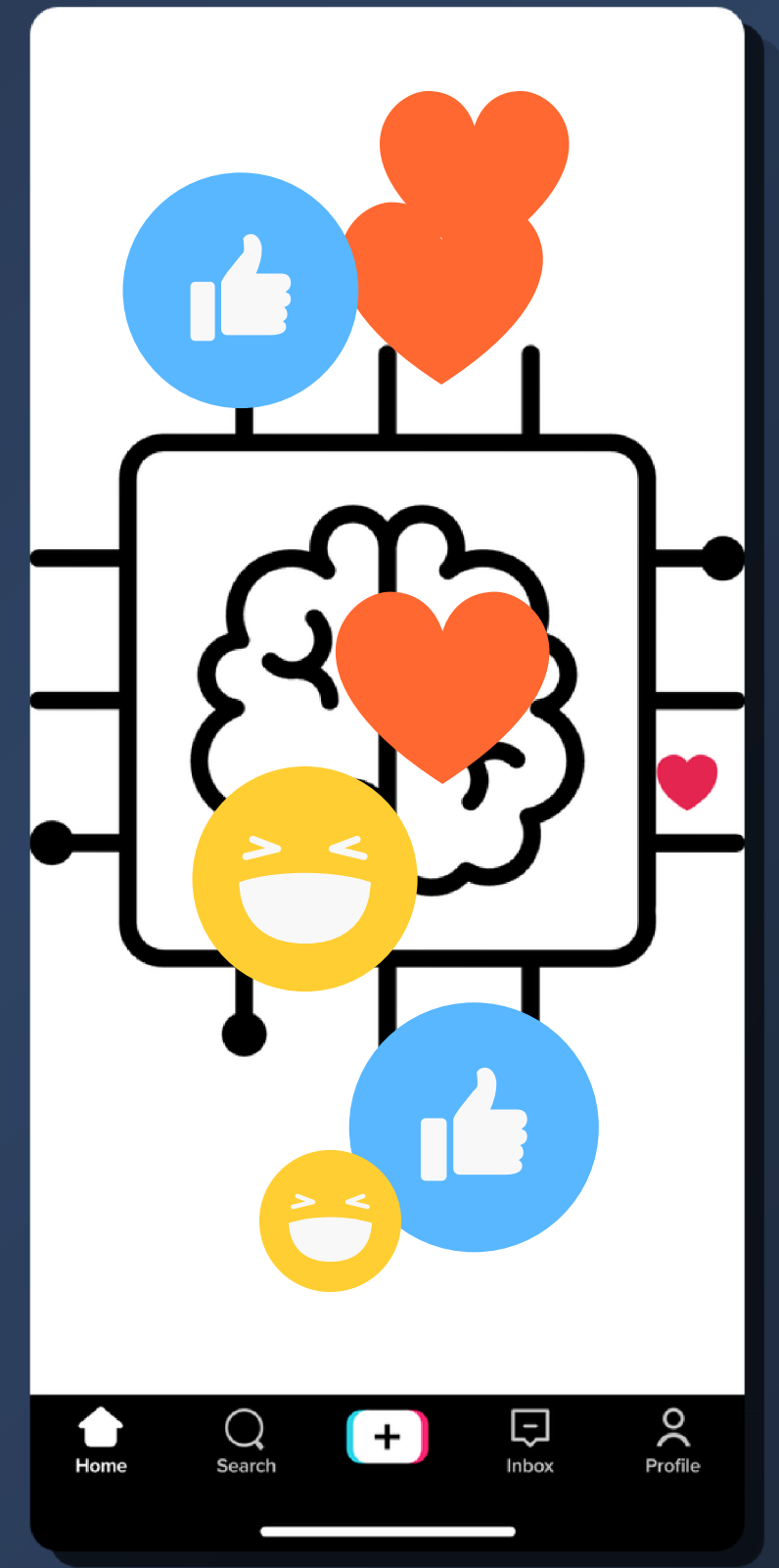
5 responses



- Completed without help
- Completed but needed help
- Couldn't complete it

Deliverable 9

Optimizing your Prototype



Optimization Backlog

Natural Language Processing

On track

Insights

Workflows 7

Task Tracker

Proj Roadmap

MoSCoW

Sprint 1

Sprint 2

sprint3

+ New view

Filter by keyword or by field

View

Title

Impact

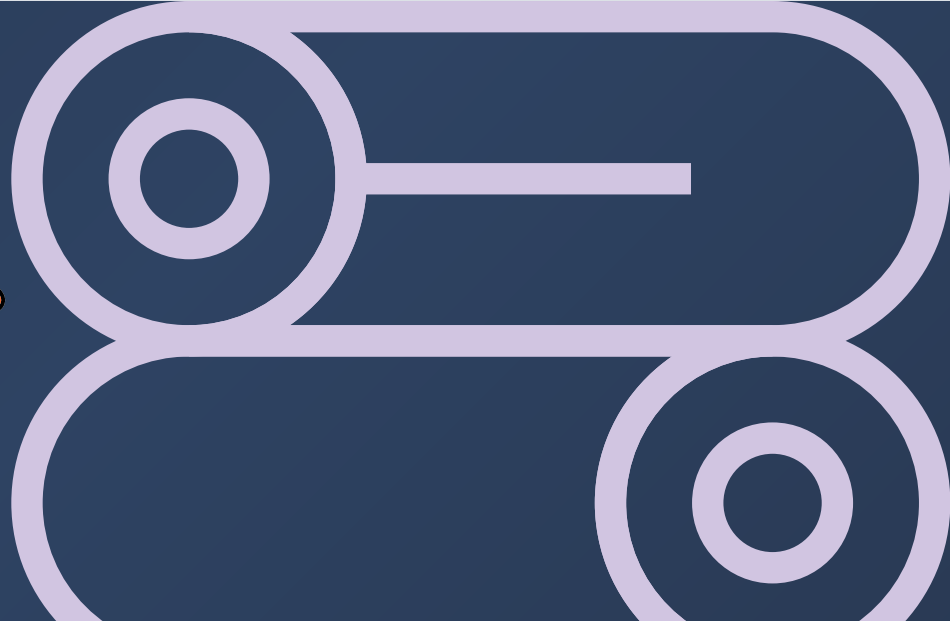
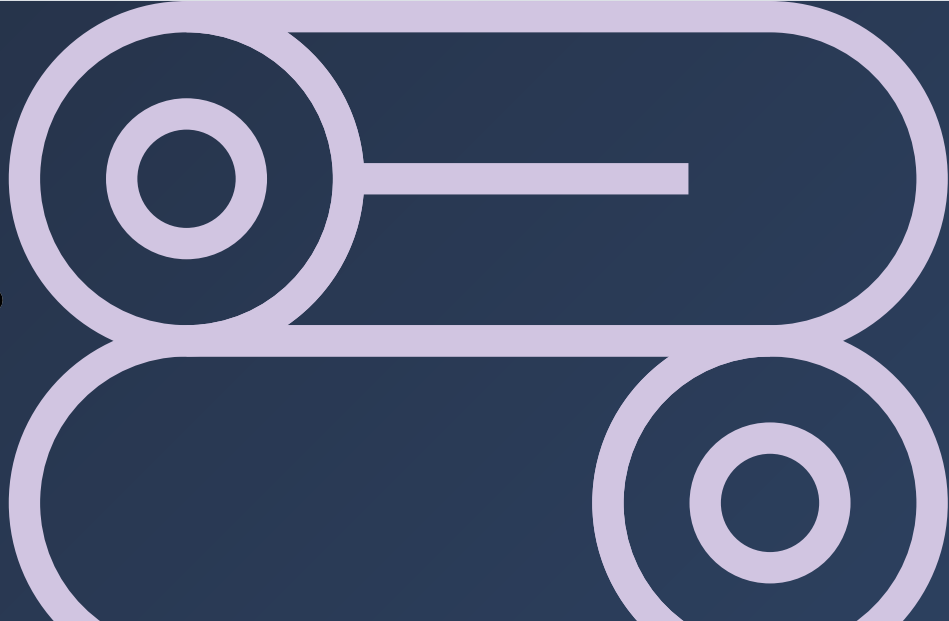
Eff...

Priority

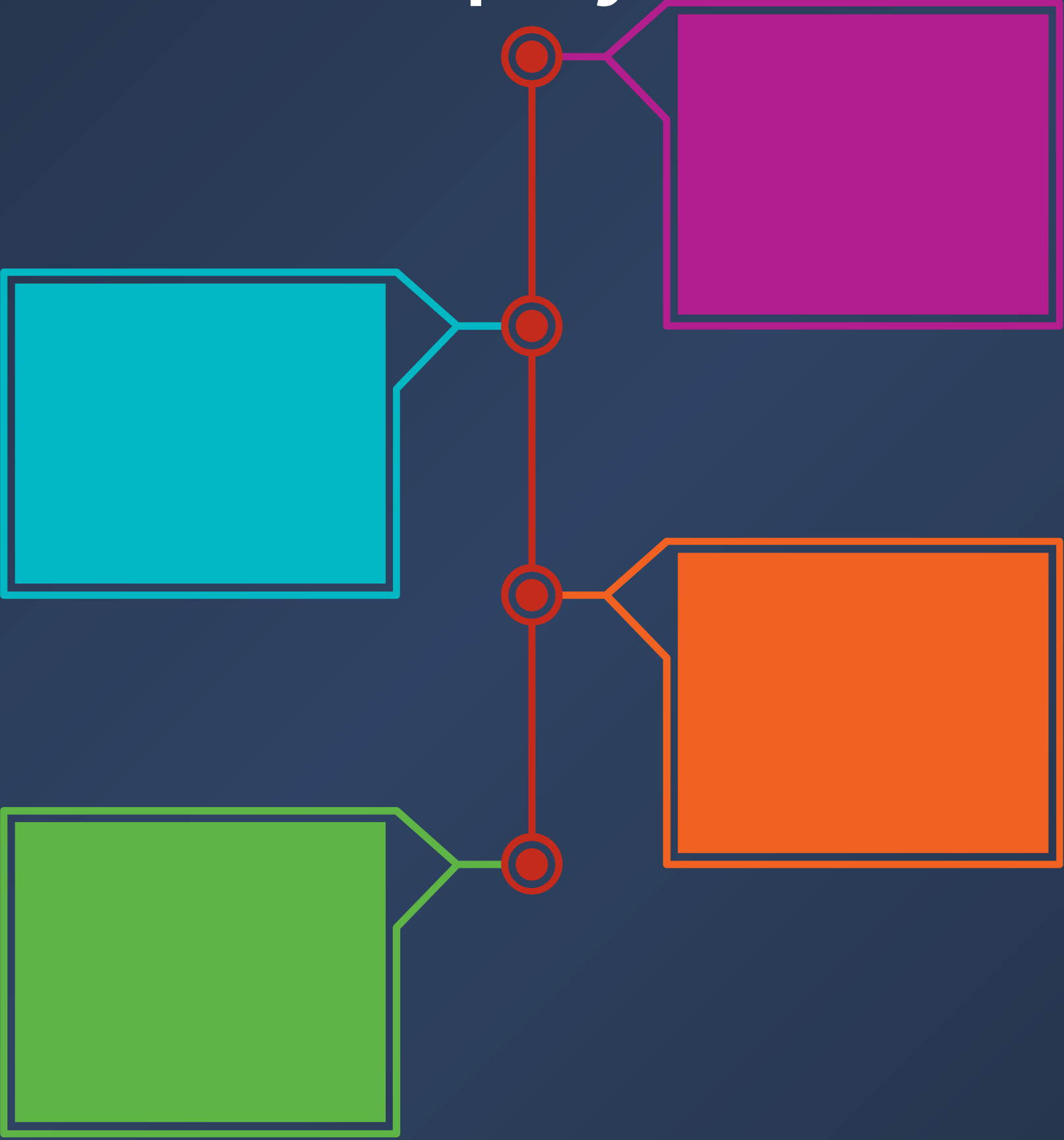
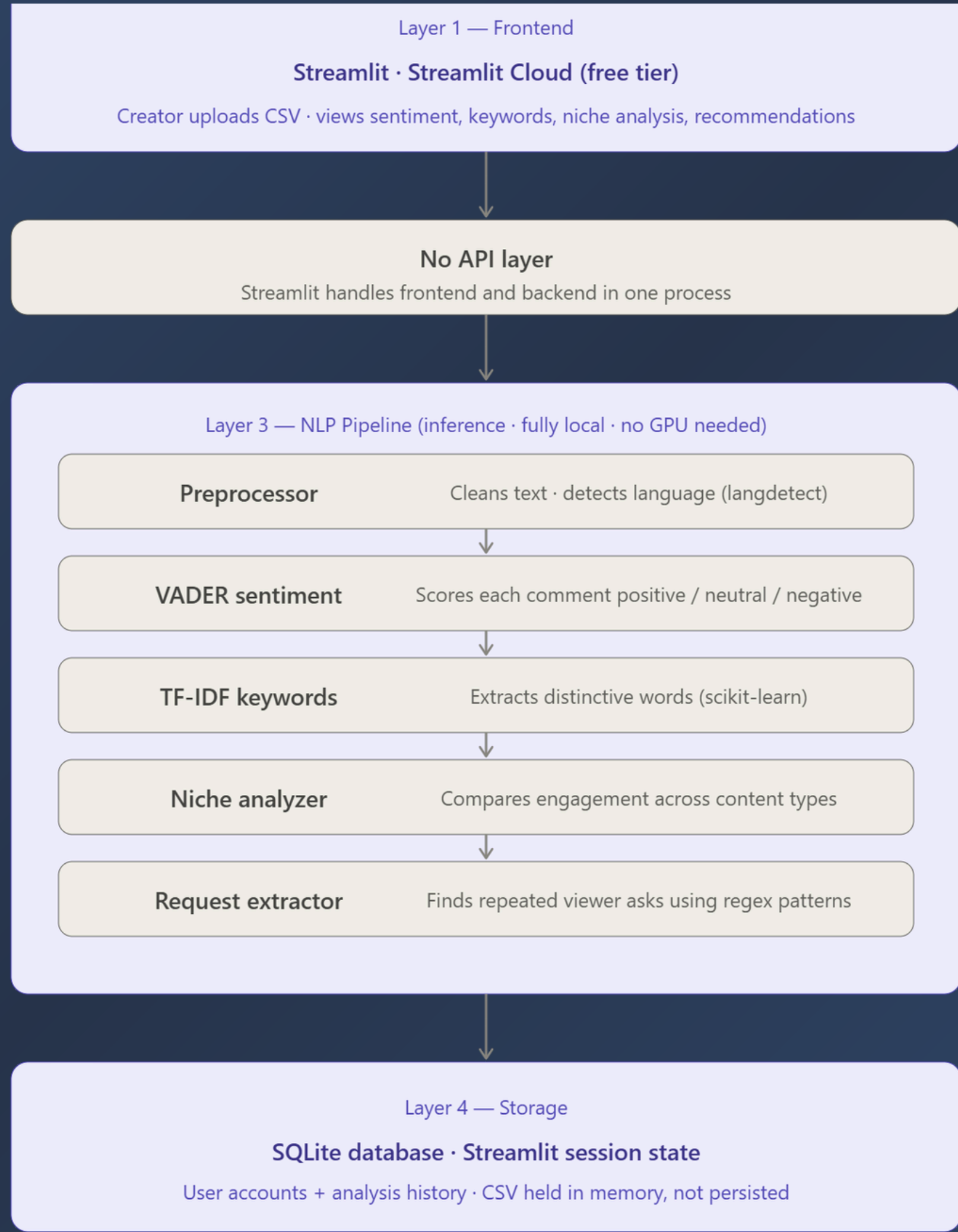
Acceptance Criterion

Sprint 4 7 Jun 19 - Jun 28 Current

24	[P1] Fix langdetect false positives for short ASCII-only comments #58	HIGH	S	P1	Non-English exclusion drops from 22.0% to 12.0% (142 short comments recovered)
25	[P1] Add custom VADER lexicon entries for TikTok hype slang #59	HIGH	S	P1	Macro F1 stays >= 94.1%; hype-slang words (hard/crazy/sick/dead) score positive o...
26	[P1] Automate regression test via test_harness.py on every git push (GitHub Action... #60	HIGH	S	P1	CI runs test_harness.py on every push; exits 1 if any quality flag fires
27	[P2] Show keyword cluster match reason in Dashboard #61	MEDIUM	M	P2	Each keyword cluster shows the seed token that triggered the assignment
28	[P2] Expand language support beyond German (Spanish, French, Portuguese) #62	HIGH	L	P2	Spanish, French, Portuguese translated and processed; total exclusion rate below 12%
29	[P2] Cache analysis result per file hash to skip re-running pipeline on same upload #63	MEDIUM	S	P2	Same file re-upload completes in under 0.1s with no pipeline re-run
30	[P2] Flag sarcasm-marker comments as low-confidence in the Dashboard #64	LOW	M	P2	5 sarcasm-risk comments display a low-confidence label in the Dashboard

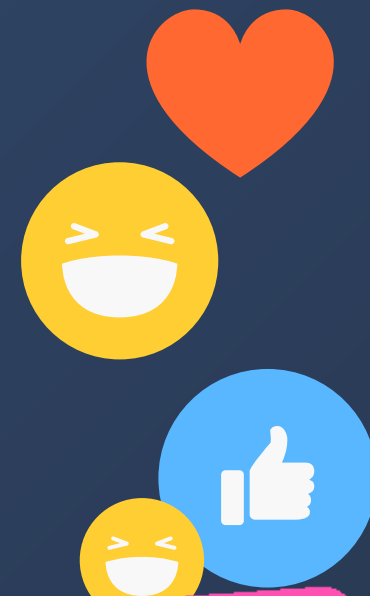


Production Deployment Plan



Monitoring and Observability

Metric	Target	Alert Threshold
p95 analysis latency (ms)	< 500 ms	Alert if > 1,000 ms
Non-English exclusion rate (%)	< 22%	Alert if > 30%
Upload error rate (%)	< 2%	Alert if > 5%



Task 4

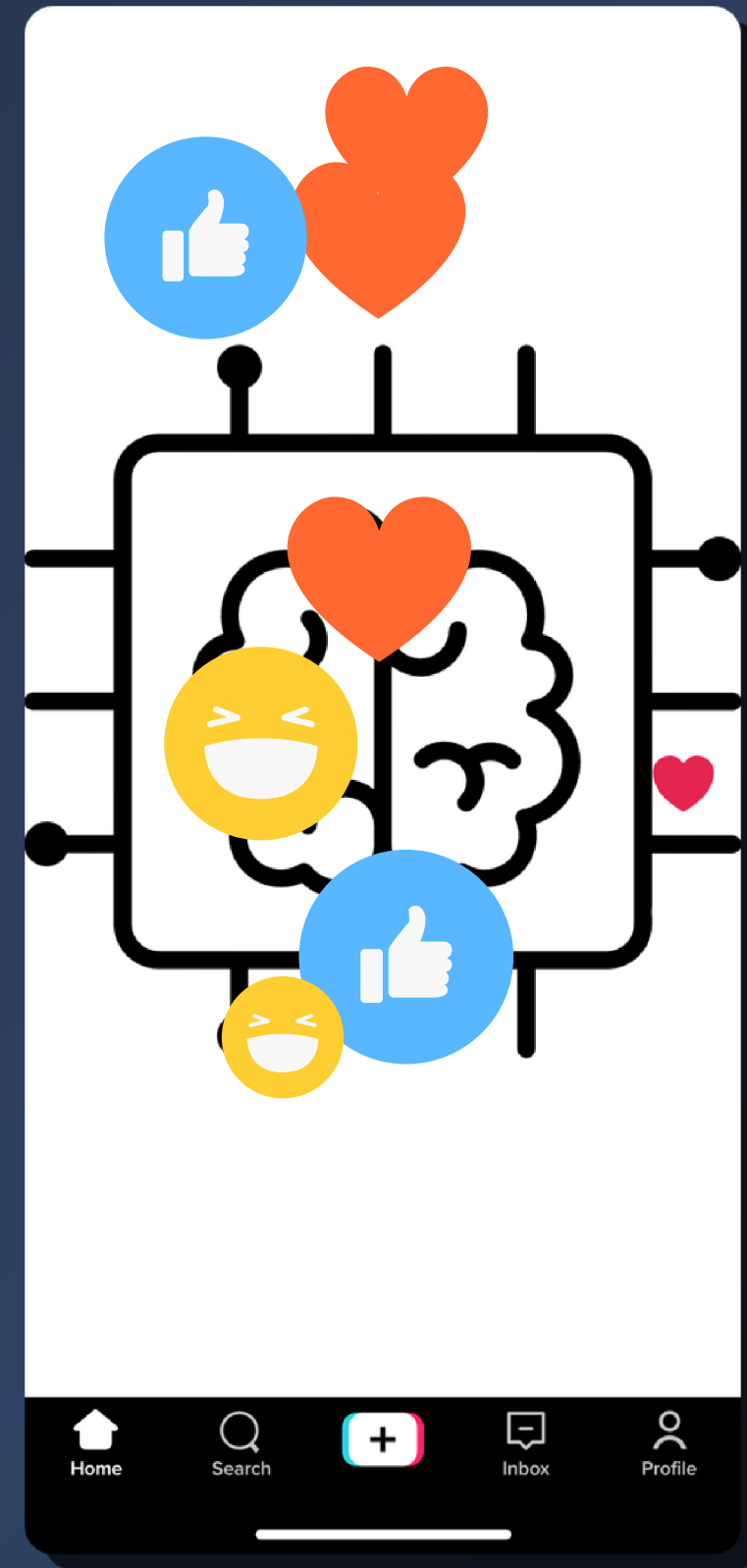
Change	Metric Before	Metric After	Verdict
Add TikTok hype slang to VADER lexicon (hard, crazy, sick, dead, fire → positive)	2/3 slang comments correctly scored	3/3 (100%)	Keep -- "This community is crazy" was scored negative, now positive; no regression on format test set (F1 stays 94.1%)
Trust short ASCII-only comments (≤3 tokens) as English instead of dropping them	Non-English exclusion rate 22.0%	16.20%	Keep -- 71 comments recovered with no accuracy cost



task 5

Item	Acceptance Criterion	Result
Fix langdetect false positives: trust short ASCII-only comments as English	Exclusion rate drops from 22.0% to $\leq 12.0\%$	16.2% -- PARTIAL PASS (71 comments recovered; emoji-containing short comments still excluded)
Add TikTok hype slang to VADER lexicon (hard, crazy, sick, dead, fire)	Macro F1 stays $\geq 94.1\%$; hype-slang comments score positive	F1 = 94.1% maintained; "This community is crazy" correctly rescored negative $\rightarrow$ positive -- PASS
Automate regression test via GitHub Actions CI (.github/workflows/regression.yml)	CI runs test_harness.py on every push; exits 1 if any flag fires	Workflow created; test_harness.py ran clean across 10 datasets, 0 flags -- PAS

Final Presentation

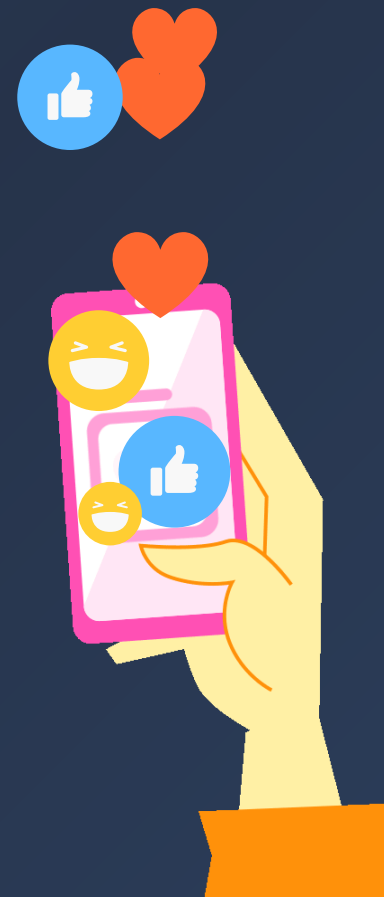


Task 1: The Hook and Narrative Arc



Persona

Amara, 21, student and lifestyle content creator with 2,300 TikTok followers. She posts three times a week across food, travel, and daily life content. Her pain point: she gets 50 to 100 comments per video but reads them one by one on her phone and can never tell which content type her audience actually prefers.



Before

Every Sunday evening Amara scrolls through 200+ comments across the week's videos, trying to spot patterns manually. She notices her food video got more comments than her vlog but cannot tell if that gap is real or just one lucky video. She posts whatever feels right that week and hopes for the best. Three months in, her growth has stalled and she does not know why.

After + Elevator Pitch

Amara uploads one CSV. In under a second she sees her Food and Cooking videos average 26 comments each versus 7 for Lifestyle -- a 3.6x gap the data makes unmissable. The recommendations page shows 27 viewers already asked for a camping gear video.

Elevator Pitch: TikTok Creator Intelligence turns 200 unread comments into one clear answer about what to post next -- so small creators can grow with data, not guesswork.

What to Post More

Double Down on Food & Cooking

Why: Your Food & Cooking videos average 23.0 (10,328 vs 1,406 on average).

Tip: Make Food & Cooking your main lane. Ne

What to Improve ↻

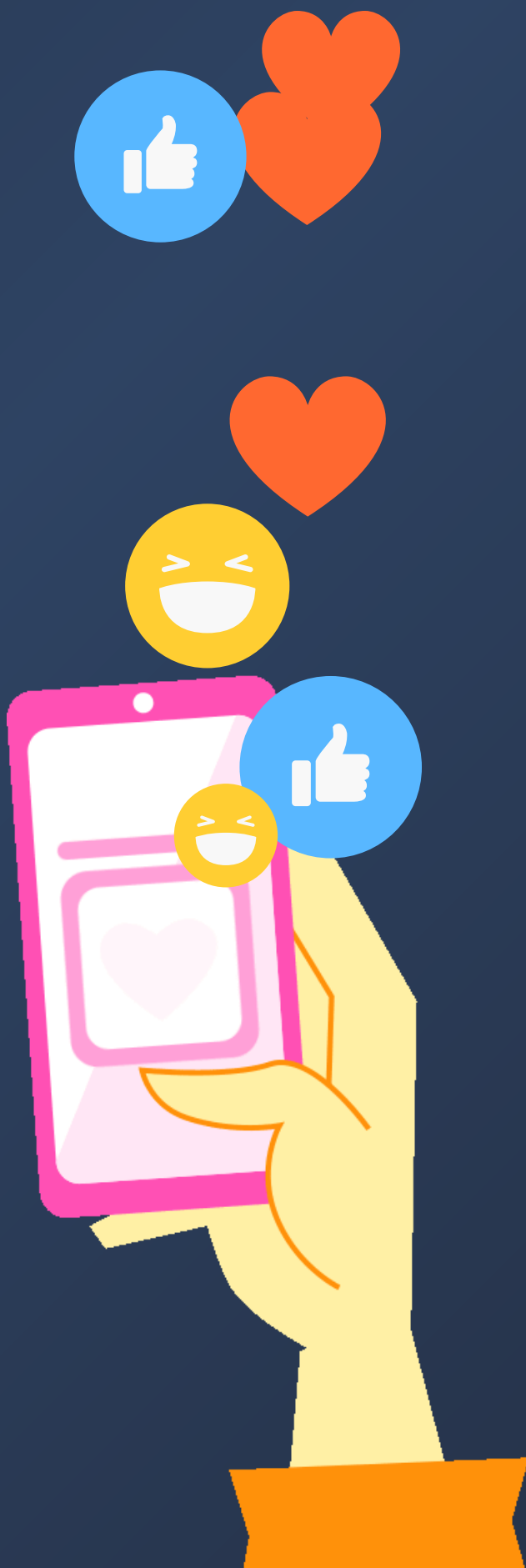
High Negative Score — Check What It Rea

Why: 12.6% of all your comments score negativ

Tip: Read a sample before changing anything; real criticism, look for the recurring complain

Task 2: The Failure Post-Mortem

Aspect	TikTok Creator Intelligence
What failed	The recommendation system generated meaningless advice citing "content" as the top keyword -- producing outputs like "viewers are asking for content type of content"
When discovered	D8 user evaluation, Week 10 -- tester Wudh found the What to Improve section empty despite 14.4% negative sentiment, and keyword clusters showed platform words instead of topics
Root cause	TF-IDF scored generic platform words ("content", "video", "channel") highest because they appear in almost every comment -- making them statistically distinctive but meaninglessly generic; no filter existed to block them
Impact on D8 metrics	"The analysis felt true" user rating: 3.0 / 5; trust gap in keyword categorization flagged by 3 of 4 testers; negativity threshold set at 15% caused the What to Improve section to silently produce nothing for Wudh's 14.4% negative dataset
Pivot and outcome	Added GENERIC_FILLER word list to nlp/keywords.py blocking platform words from TF-IDF; rebuilt recommendation generator to only fire a card when measured evidence exists and every card cites actual numbers. Result: Wudh's issue resolved, no later tester reported generic recommendations, SUS rose to 87.5 / A+

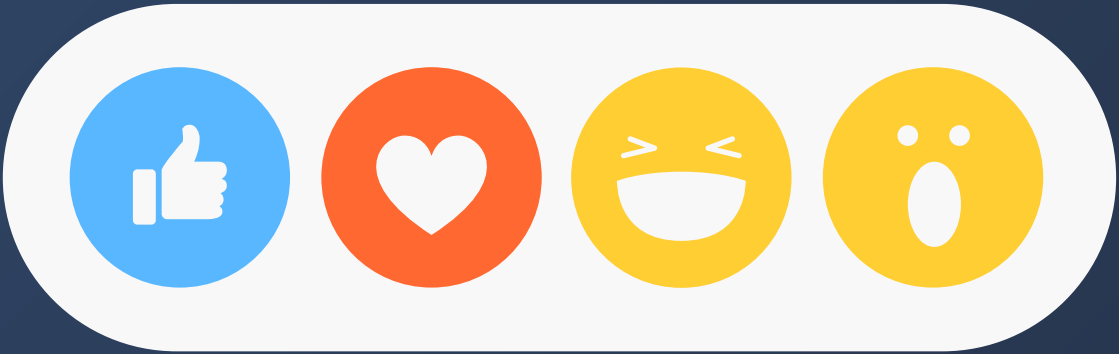


Task 3 The Live-Demo Storyboard				
#	Screen State	Narration Script	Latency Risk	Backup Action
1	App open on Upload Data page, no file loaded	"Meet Amara -- 2,300 followers, posts three times a week, reads every comment on her phone and still cannot tell what is working. This is the tool I built to fix that."	None	Pre-loaded screenshot of the upload page
2	Drag and drop comments CSV into the uploader	"She exports her TikTok comments as a CSV -- one file, that is all the system needs. I am uploading my own real data from @ichbinnelo, 1,211 comments scraped across two years."	None	File already staged, single click to upload
3	Upload videos CSV into the optional uploader	"The second file is optional -- video metadata. When it is there, the system unlocks niche comparison. Without it you still get sentiment and keywords."	None	Skip this step and note the feature verbally
4	Click Run Analysis -- spinner visible for under 1 second	"One click. The pipeline runs entirely locally -- no API call, no GPU, no cost. VADER scores every comment, TF-IDF extracts the keywords, the niche analyzer compares engagement across content types."	< 0.5 s	"The analysis already finished -- watch how fast that was."
5	Dashboard page loads -- sentiment cards and charts visible	"The dashboard shows the full picture instantly. 72% positive, 14% negative. The bar chart shows which words are most distinctive in her comments -- not generic words like 'content', actual topics."	None	Pre-run screenshot of Dashboard saved on desktop
6	Navigate to Recommendations page -- Double Down card visible	"This is the part that matters. Her Food and Cooking videos average 26 comments each versus 7 for Lifestyle -- a 3.6x gap. The system spotted that without being told to look for it."	None	Read the card text aloud from memory
7	Scroll to Suggested Content Ideas section	"27 viewers asked for a camping gear video. 22 asked where a specific trail was. These are not guesses -- they are exact phrases counted across the comment section."	None	Pre-prepared list of top requests on a backup slide
8	Switch to browser tab showing GitHub Actions CI	"Every time I push code, a regression test runs automatically against 37 manually labelled comments. If accuracy drops, the push is blocked. The system protects itself."	None	Screenshot of passing CI run
9	Return to Dashboard, point to sentiment numbers	"D8 measured 91.9% accuracy and 94.1% Macro F1 against manually reviewed ground truth -- beating the naive baseline by 37 points. Small creator, real data, zero cloud cost."	None	README S8 table open in second tab

Task 4 The Lessons Learned Matrix

	Expected	Surprising
Technical	VADER works well on emoji-rich social media text -- it was designed for this, and the 91.9% accuracy on real TikTok comments confirmed the architecture choice made in D4. TF-IDF extracts keywords reliably but requires domain-specific stop words -- generic platform words ("content", "video") score highest and must be manually blocked.	Stripping emojis to "clean" the text hurt accuracy by 5.4 points -- the emoji is often the entire sentiment signal on TikTok. A 12-word GENERIC_FILLER list fixed what felt like a fundamental model problem -- the generic recommendations bug that drove the full 9-task refactor was a data filtering issue, not a model issue.
Procedural	User testing surfaces problems developers cannot see -- testers immediately found the keyword cluster opacity and generic recommendations issues that code review missed. Building an end-to-end system takes longer than isolated modules -- connecting preprocessing to sentiment to recommendations introduced bugs that never appeared in unit testing.	One tester independently found the exact VADER hype slang failure that the D8 error analysis had predicted theoretically - user testing confirmed a documented bug without being told to look for it. The hardest deliverable was evaluation (D8), not building (D7) -- measuring quality rigorously requires more discipline than writing the code.

Most Valuable Lesson: "A pre-trained model is only as good as the data you feed it -- VADER was accurate out of the box, but every real improvement came from fixing what went in (emoji preservation, slang lexicon, generic word filtering), not from changing the model itself.





you can visit the app here

<https://tiktok-creator-intelligence.streamlit.app>

